

第2章 Vue.js开发基础

学习目标/Target

掌握单文件组件，能够创建并使用单文件组件

掌握数据绑定，能够定义数据并将数据渲染到页面中

掌握内容渲染指令，能够灵活运用v-text和v-html指令将字符串渲染到页面中

掌握属性绑定指令，能够灵活运用v-bind指令给目标元素的属性动态绑定值

掌握事件绑定指令，能够灵活运用v-on指令给目标元素绑定事件

学习目标/Target

掌握双向数据绑定指令，能够灵活运用v-model指令在表单元素上实现数据的双向绑定

掌握条件渲染指令，能够灵活运用v-if、v-show指令根据不同的条件渲染不同的标签

掌握列表渲染指令，能够灵活运用v-for指令将数组、对象等中的数据渲染到页面中

掌握事件对象，能够灵活运用事件对象获取和修改DOM元素的属性

学习目标/Target

熟悉事件修饰符，能够阐述常见的事件修饰符

掌握计算属性，能够灵活运用计算属性实时监听数据的变化，当所依赖的数据发生变化时重新计算值

掌握侦听器，能够灵活运用侦听器监听数据的变化并进行相应的操作

掌握样式绑定，能够灵活运用v-bind绑定class和style属性实现元素样式的设置

章节概述/ Summary

在搭建好Vue开发环境后，要想使用Vue开发实际项目，必须先学习Vue基础知识。只有掌握Vue框架的基础知识后，才能根据实际需求游刃有余地进行项目开发。本章将详细讲解Vue开发基础。

目录/Contents

2.1

单文件组件

2.2

数据绑定

2.3

指令

2.4

事件对象

2.5

事件修饰符

目录/Contents

2.6

计算属性

2.7

侦听器

2.8

样式绑定

2.9

阶段案例——学习计划表

2.1

单文件组件

2.1 单文件组件



掌握单文件组件，能够创建并使用单文件组件

2.1 单文件组件

什么是单文件组件?



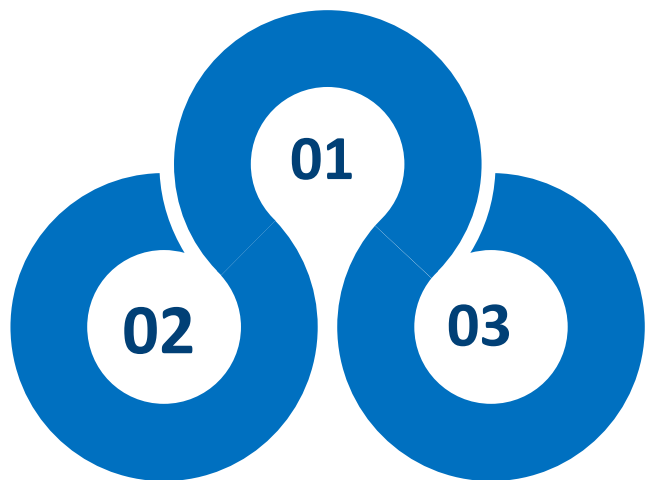
2.1 单文件组件



在第1章中使用Vite创建Vue项目后，目录结构中包含一些扩展名为`.vue`的文件，每个`.vue`文件都可用来定义一个单文件组件。
Vue中的单文件组件是Vue组件的文件格式。

2.1 单文件组件

每个单文件组件由模板、样式和逻辑3个部分构成。



1

模板

模板用于搭建当前组件的DOM结构，其代码写在<template>标签中。

2

样式

样式是指通过CSS代码为当前组件设置样式，其代码写在<style>标签中。

3

逻辑

逻辑是指通过JavaScript代码处理组件的数据与业务，其代码写在<script>标签中。

2.1 单文件组件

演示如何定义一个基本的单文件组件。

```
<template>
  <!-- 此处编写组件的结构 -->
</template>
<script>
/* 此处编写组件的逻辑 */
</script>
<style>
/* 此处编写组件的样式 */
</style>
```

2.1 单文件组件

演示单文件组件的使用方法

步骤1

打开命令提示符，切换到D:\vue\chapter02目录，在该目录下执行如下命令，创建项目。

```
yarn create vite vue-demo --template vue
```

步骤2

项目创建完成后，执行如下命令进入项目目录，启动项目。

步骤3

```
cd vue-demo
```

步骤4

```
yarn
```

```
yarn dev
```

步骤5

项目启动后，可以使用URL地址<http://127.0.0.1:5173/>进行访问。

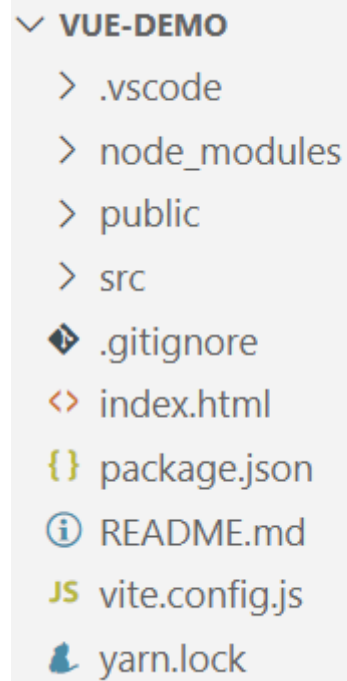
2.1 单文件组件

演示单文件组件的使用方法

步骤1

使用VS Code编辑器打开D:\vue\chapter02\vue-demo目录。

步骤2



▼ VUE-DEMO

- > .vscode
- > node_modules
- > public
- > src
- 📄 .gitignore
- <> index.html
- { } package.json
- 📄 README.md
- JS vite.config.js
- 📄 yarn.lock

步骤3

步骤4

步骤5

2.1 单文件组件

演示单文件组件的使用方法

步骤1

步骤2

步骤3

步骤4

步骤5

将src\style.css文件中的样式代码全部删除，从而避免项目创建时自带的样式代码影响本案例的实现效果。

2.1 单文件组件

演示单文件组件的使用方法

创建src\components\Demo.vue文件，该文件是Demo组件。

步骤1

步骤2

步骤3

步骤4

步骤5

```
<template>
  <div class="demo">Demo组件</div>
</template>
<style>
.demo {
  font-size: 22px;
  font-weight: bold;
}
</style>
```

2.1 单文件组件

演示单文件组件的使用方法

步骤1

修改src\main.js文件，切换页面中显示的组件。

步骤2

```
import App from './components/Demo.vue'
```

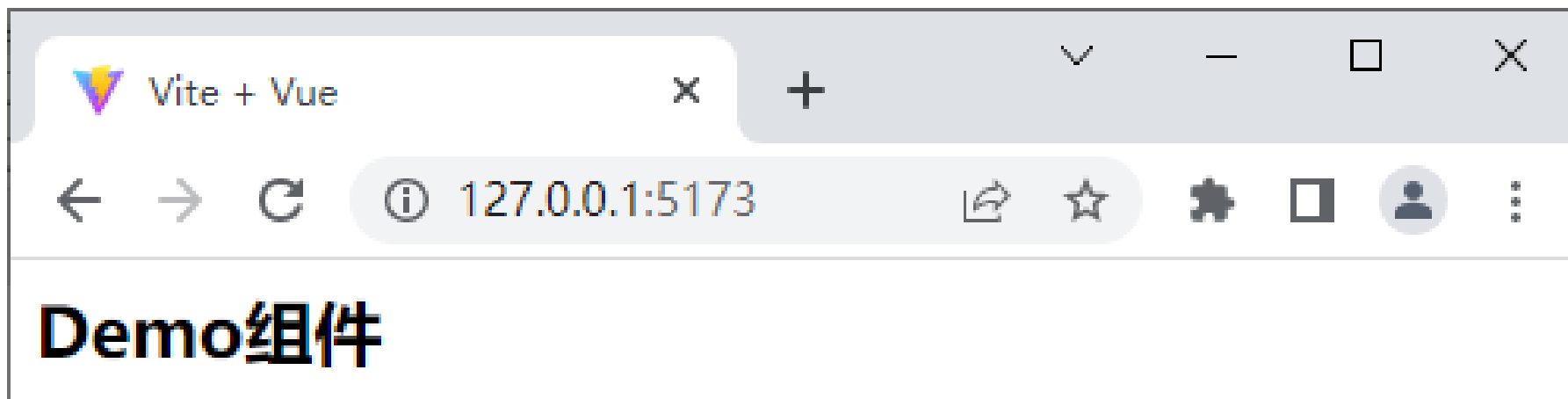
步骤3

步骤4

步骤5

2.1 单文件组件

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，Demo组件的页面效果如下图所示。



2.2

数据绑定

2.2.1 初识数据绑定



掌握数据绑定，能够定义数据并将数据渲染到页面中

2.2.1 初识数据绑定



Vue通过数据绑定实现了数据与页面相分离，从而实现了数据驱动视图的效果。即将页面中的数据分离出来，放到组件的<script>标签中，通过程序操作数据，并且当数据改变时，页面会自动发生改变。

2.2.1 初识数据绑定

数据绑定分为定义数据和输出数据。

定义数据

由于数据和页面是分离的，在实现数据显示之前，需要先在`<script>`标签中定义组件所需的数据。

输出数据

Vue为开发者提供了Mustache语法（又称为双大括号语法），将数据输出到页面中。



2.2.1 初识数据绑定

1. 定义数据

定义数据的语法格式如下。

```
<script>
export default {
  setup() {
    return {
      数据名: 数据值,
      .....
    }
  }
}
</script>
```


2.2.1 初识数据绑定

1. 定义数据

1: Message.vue

```
<template>
  {{ message }}
</template>

<script setup>
let message = '不积跬步，无以至千里';
</script>
```

2.2.1 初识数据绑定

为了让代码更简洁，Vue 3提供了`setup`语法糖（Syntactic Sugar），使用它可以简化上述语法，提高开发效率。

使用`setup`语法糖来定义数据的语法格式如下。

```
<script setup>
const 数据名 = 数据值
</script>
```

VS.

```
<script>
export default {
  setup() {
    return {
      数据名: 数据值,
      .....
    }
  }
}
</script>
```

2.2.1 初识数据绑定

2. 输出数据

Vue为开发者提供了Mustache语法（又称为双大括号语法），使用该语法时相当于在模板中放入占位符，其语法格式如下。

{{ 数据名 }}



当页面渲染时，模板中的{{ 数据名 }}会被替换为实际的数据

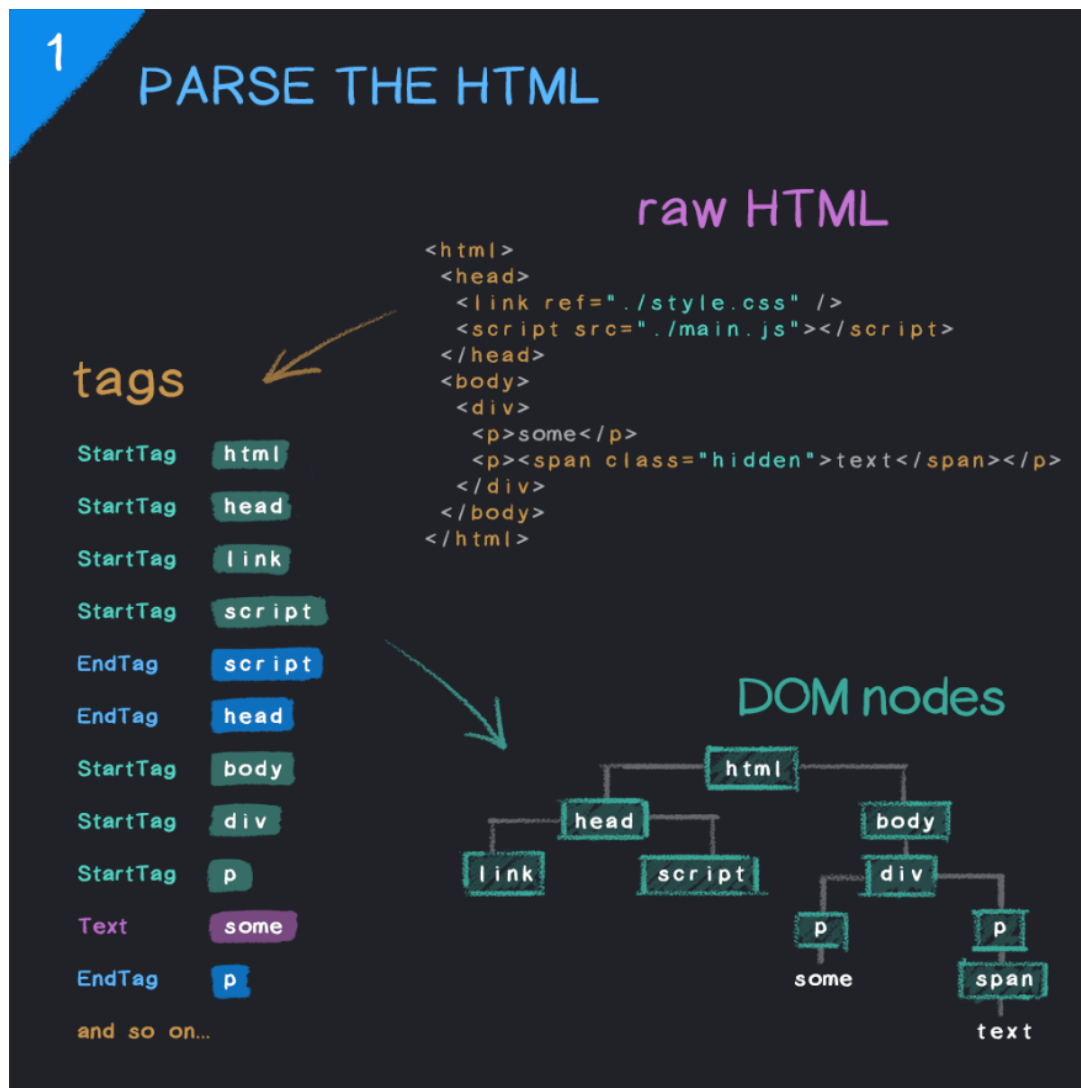
2.2.1 初识数据绑定

在 HTML 中，**渲染 (Rendering)** 指的是浏览器将接收到的 HTML、CSS 和 JavaScript 代码转换成用户在屏幕上看到的网页的过程。

1. 开始解析HTML
2. 获取外部资源
3. 解析 CSS 并构建CSSOM
4. 执行 JavaScript
5. 合并 DOM 和 CSSOM 以构造渲染树
6. 计算布局和绘制

1.开始解析HTML

当浏览器通过网络接收页面的HTML数据时，它会立即设置解析器将HTML转换为文档对象模型(DOM)。



2. 获取外部资源

当解析器遇到外部资源（如CSS或JavaScript文件）时，解析器将提取这些文件。

2

FETCH EXTERNAL RESOURCES

CSS blocks rendering

JavaScript blocks parser...

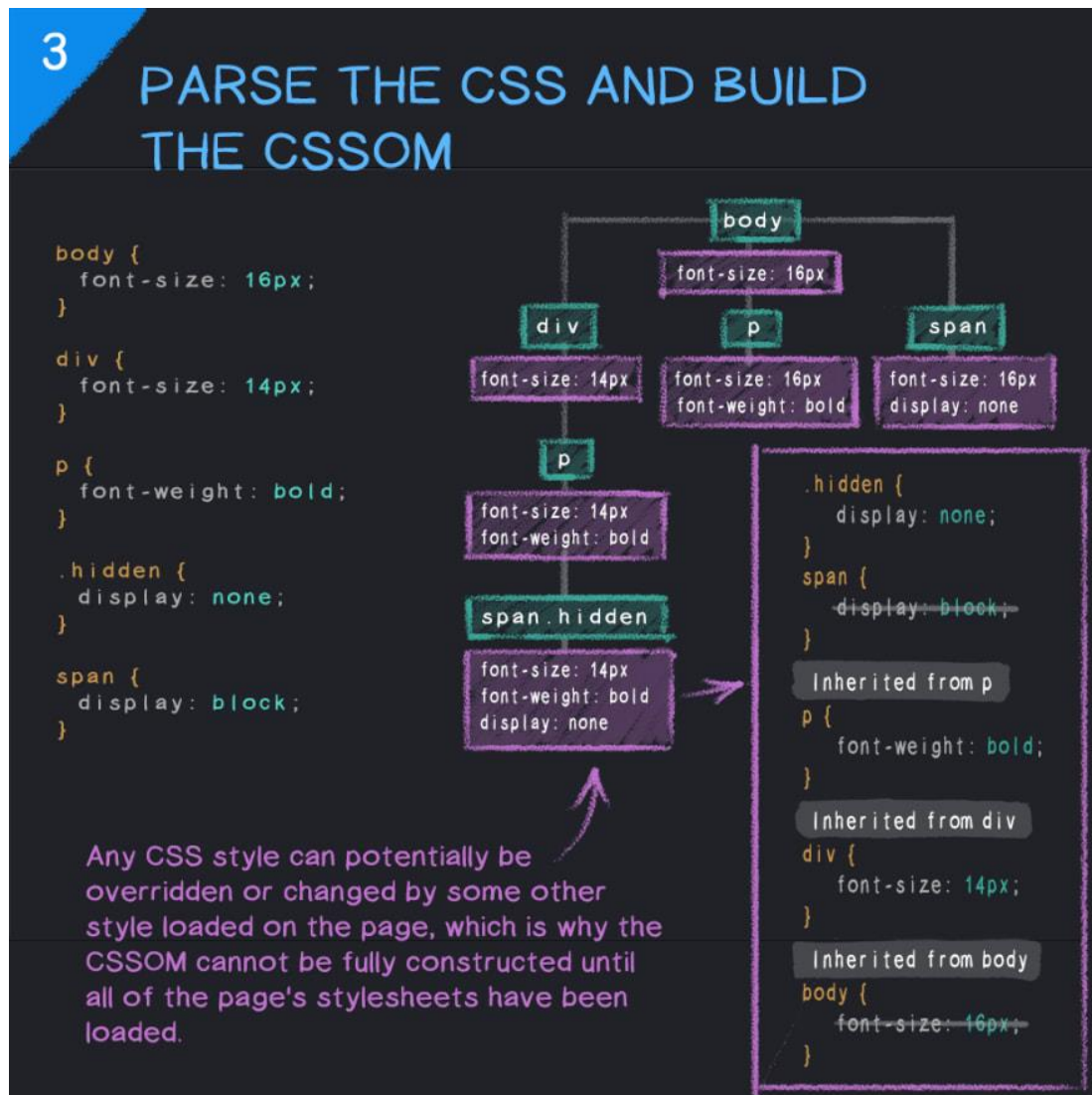
← unless! →

defer waits until parser is finished

async executes as soon as it loads

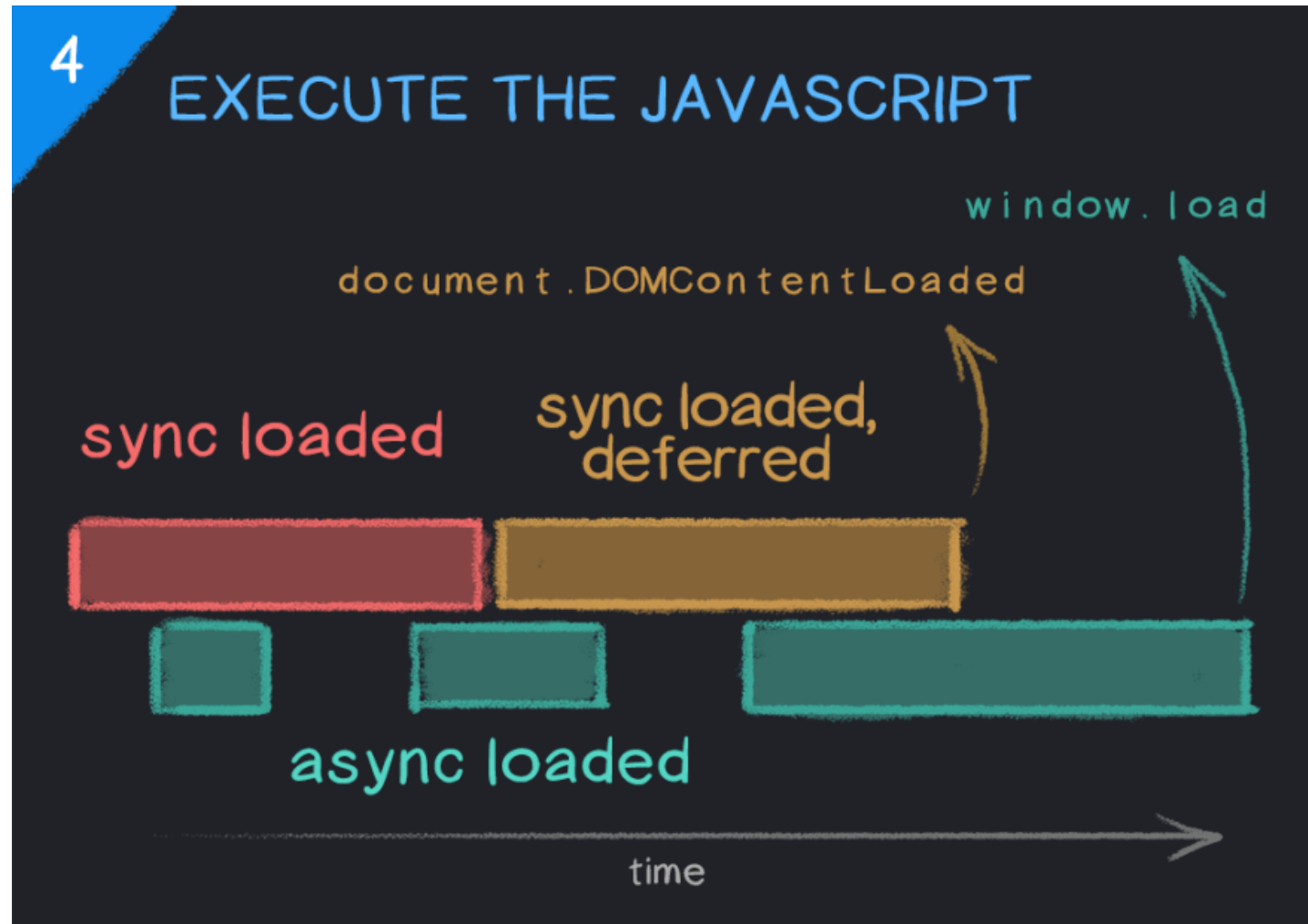
3.解析CSS并构建CSSOM

与HTML文件和DOM相似，加载CSS文件时，必须将它们解析并转换为树-这次是CSSOM。



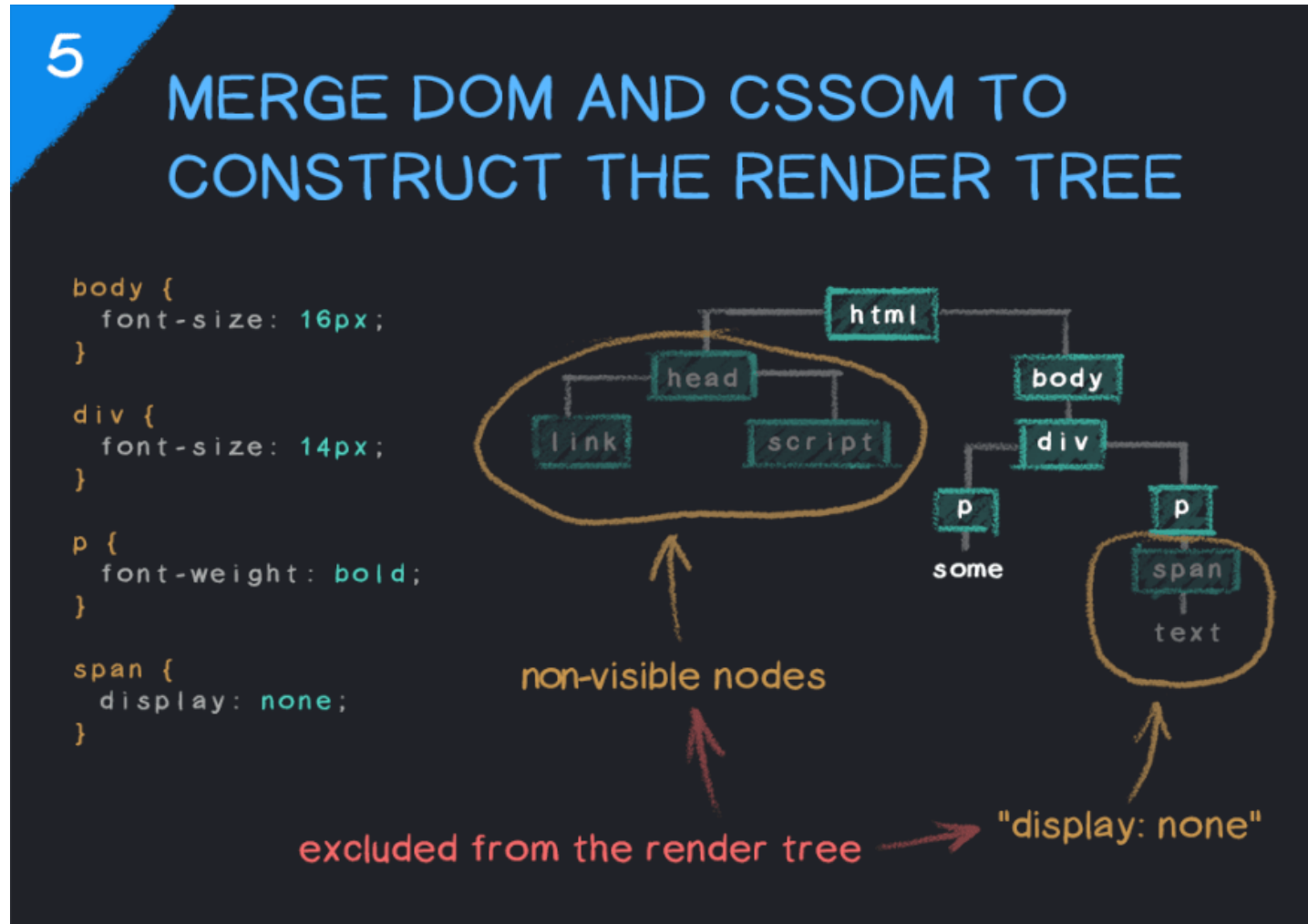
4. 执行JavaScript

不同的浏览器有不同的 JS 引擎来执行此任务。



5.合并DOM和CSSOM 构建渲染树

渲染树是DOM和CSSOM的组合，表示将要渲染到页面上的所有内容。



6. 计算布局和绘制

现在有了完整的渲染树，浏览器知道了要渲染什么，但是在哪里渲染？

6 CALCULATE LAYOUT AND PAINT

```
<html>
<head>
  <link ref="./style.css" />
</head>
<body>
  <div>
    <p>some</p>
    <p><span>text</span></p>
  </div>
</body>
</html>
```

html

body

div

some p

p

span text

let's add some layout styles...

```
* {
  padding: 25px;
}

html {
  height: 100%;
}

body {
  width: 50%;
}

div {
  height: 200px;
}

p {
  width: 50%;
}

span {
  position: absolute;
  right: 25px;
}
```

2.2.1 初识数据绑定

在Mustache语法中还可以将表达式的值作为输出内容。表达式的值可以是字符串、数字等类型，示例代码如下。

```
{{ 'Hello Vue.js' }}  
  
{{ number + 1 }}  
  
{{ obj.name }}  
  
{{ ok ? 'YES' : 'NO' }}  
  
{{ '<div>HTML标签</div>' }}
```

2.2.1 初识数据绑定

演示数据绑定的实现

步骤1

创建src\components\Message.vue文件。

```
<template>{{ message }}</template>  
<script setup>  
const message = '不积跬步，无以至千里'  
</script>
```

步骤2

用于在模板中输出message

2.2.1 初识数据绑定

演示数据绑定的实现

步骤1

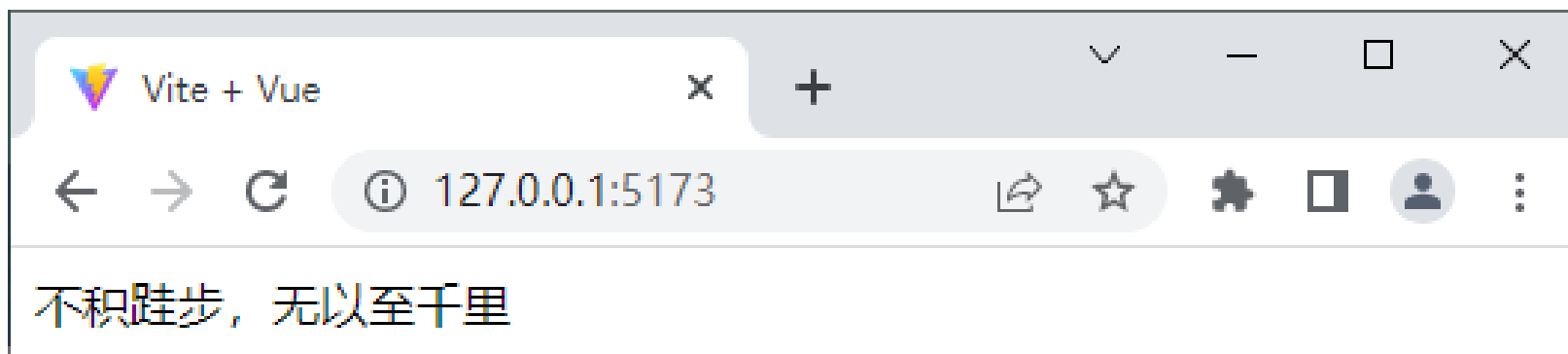
修改src\main.js文件，**切换**页面中**显示**的**组件**。

```
import App from './components/Message.vue'
```

步骤2

2.2.1 初识数据绑定

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，数据绑定的页面效果如下图所示。



2.2.1 初识数据绑定



多学一招：将Vue引入HTML页面中

前面学习的方式是通过Vite创建一个新的项目，这个项目中不仅有Vue，而且整合了一系列辅助开发的工具。

其实Vue的使用方式非常灵活，它还提供了另一种使用方式，就是将Vue引入HTML页面中。但这种方式只能使用Vue的核心功能，所以只适合开发一些简单的页面，在实际开发中一般不用这种方式。

2.2.1 初识数据绑定

 Vue.js

搜索

文档  API 演练场 生态系统  关于  赞助 专家  新

API 风格偏好

选项式 ☐ 组合式 

开始

简介

快速上手

基础

创建一个应用

模板语法

响应式基础

计算属性

通过 CDN 使用 Vue

你可以借助 script 标签直接通过 CDN 来使用 Vue:

```
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
```

html

这里我们使用了 [unpkg](#), 但你也可以使用任何提供 npm 包服务的 CDN, 例如 [jsdelivr](#) 或 [cdnjs](#)。当然, 你也可以下载此文件并自行提供服务。

通过 CDN 使用 Vue 时, 不涉及“构建步骤”。这使得设置更加简单, 并且可以用于增强静态的 HTML 或与后端框架集成。但是, 你将无法使用单文件组件 (SFC) 语法。

2.2.1 初识数据绑定

演示如何将Vue引入HTML页面中

步骤1

创建D:\vue\chapter02\helloworld.html文件，在该文件中通过<script>标签引入Vue。

步骤2

步骤3

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Hello World案例</title>
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"> </script>
</head>
<body>
</body>
</html>
```

2.2.1 初识数据绑定

演示如何将Vue引入HTML页面中

步骤1

在页面中定义一个用于被Vue应用实例控制的DOM区域，使开发者可以将数据填充到该DOM区域中。

步骤2

```
<div id="app">  
  <p>{{ message }}</p>  
</div>
```

步骤3

2.2.1 初识数据绑定

演示如何将Vue引入HTML页面中

在</body>结束标签前编写代码，创建Vue应用实例。

步骤1

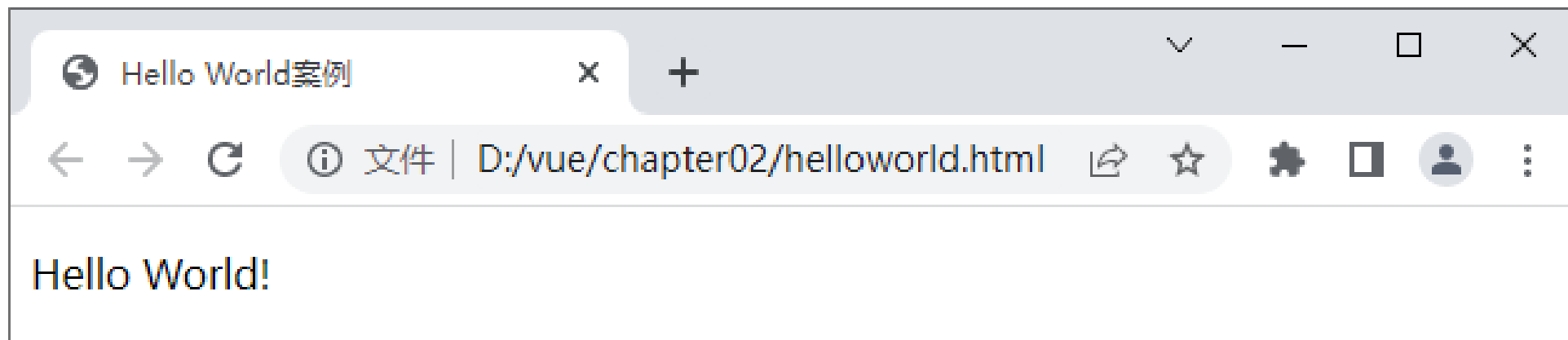
步骤2

步骤3

```
<script>
const vm = Vue.createApp({
  setup() {
    return {
      message: 'Hello World!'
    }
  }
})
vm.mount('#app')
</script>
```

2.2.1 初识数据绑定

通过浏览器访问helloworld.html，将Vue引入HTML页面的效果如下图所示。



2.2.2 响应式数据绑定



掌握响应式数据绑定，能够定义数据并将数据渲染到页面中

2.2.2 响应式数据绑定

2: 修改Message.vue → Message2.vue

```
<template>
  {{ message }}
</template>

<script setup>
let message = '不积跬步，无以至千里';
setTimeout(() => {
  console.log('更新前的message: ' + message);
  message = '长风破浪会有时，直挂云帆济沧海。';
  console.log('更新后的message: ' + message);
}, 2000)
</script>
```

打开浏览器 → 打开控制台 → 看一下有什么不同？

2.2.2 响应式数据绑定

不积跬步无以至千里



2.2.2 响应式数据绑定



如果想要实现页面中数据的更新，则需要进行响应式数据绑定，也就是将数据定义成响应式数据。

Vue为开发者提供了如下函数用于定义响应式数据。

- ref()函数
- reactive()函数
- toRef()函数
- toRefs()函数

2.2.2 响应式数据绑定

1. ref()函数

`ref()`函数用于将数据转换成响应式数据，其参数为数据，返回值为转换后的响应式数据。使用`ref()`函数定义响应式数据的语法格式如下。

```
响应式数据 = ref(数据)
```

如果需要更改响应式数据的值，可以使用如下语法格式进行修改。

```
响应式数据.value = 新值
```

2.2.2 响应式数据绑定

演示ref()函数的使用方法

步骤1

创建src\components\Ref.vue文件。

步骤2

```
<template>{{ message }}</template>
<script setup>
import { ref } from 'vue'
const message = ref('会当凌绝顶，一览众山小')
setTimeout(() => {
  message.value = '锲而不舍，金石可镂'
}, 2000)
</script>
```

2.2.2 响应式数据绑定

演示ref()函数的使用方法

步骤1

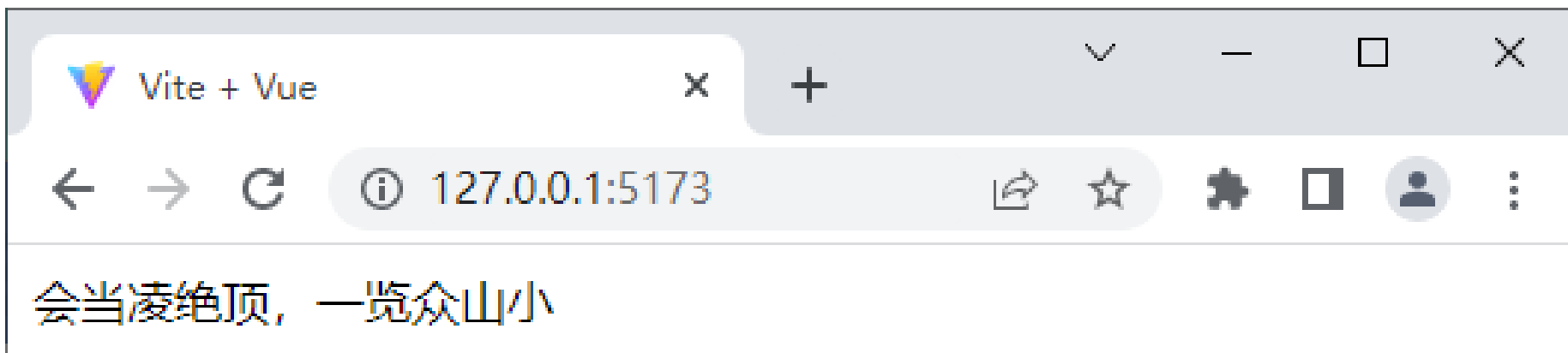
修改src\main.js文件，切换页面中显示的组件。

```
import App from './components/Ref.vue'
```

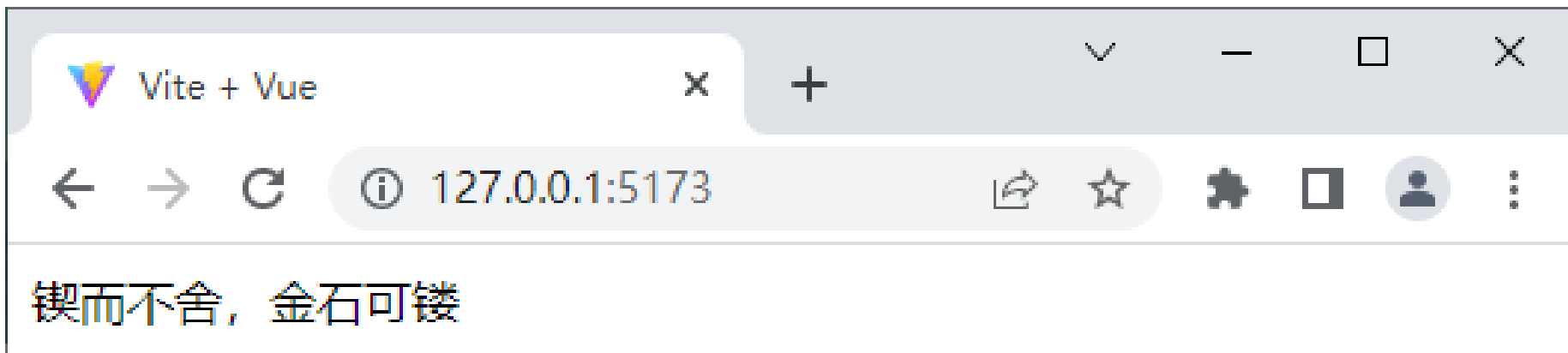
步骤2

2.2.2 响应式数据绑定

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，初始页面效果如下图所示。



等待2秒后的页面效果如下图所示。



2.2.2 响应式数据绑定

2. reactive()函数

`reactive()`函数用于创建一个响应式对象或数组，将普通的对象或数组作为参数传给该函数即可。

使用`reactive()`函数定义响应式数据的语法格式如下。

```
响应式对象或数组 = reactive(普通的对象或数组)
```

2.2.2 响应式数据绑定

演示reactive()函数的使用方法

步骤1

创建src\components\Reactive.vue文件。

步骤2

```
<template>{{ obj.message }}</template>
<script setup>
import { reactive } from 'vue'
const obj = reactive({ message: '不畏浮云遮望眼，自缘身在最高层' })
setTimeout(() => {
  obj.message = '欲穷千里目，更上一层楼'
}, 2000)
</script>
```

2.2.2 响应式数据绑定

演示reactive()函数的使用方法

步骤1

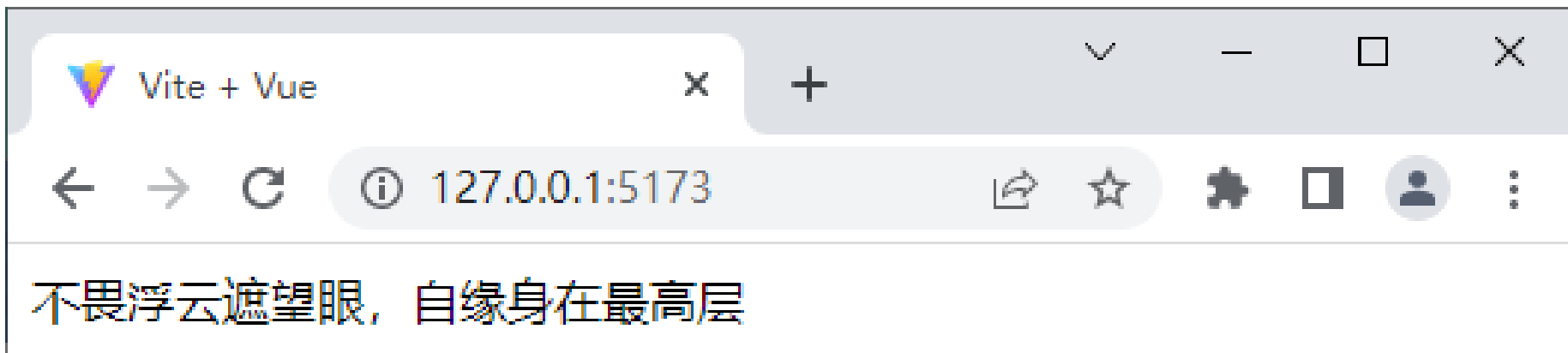
修改src\main.js文件，**切换**页面中**显示**的**组件**。

```
import App from './components/Reactive.vue'
```

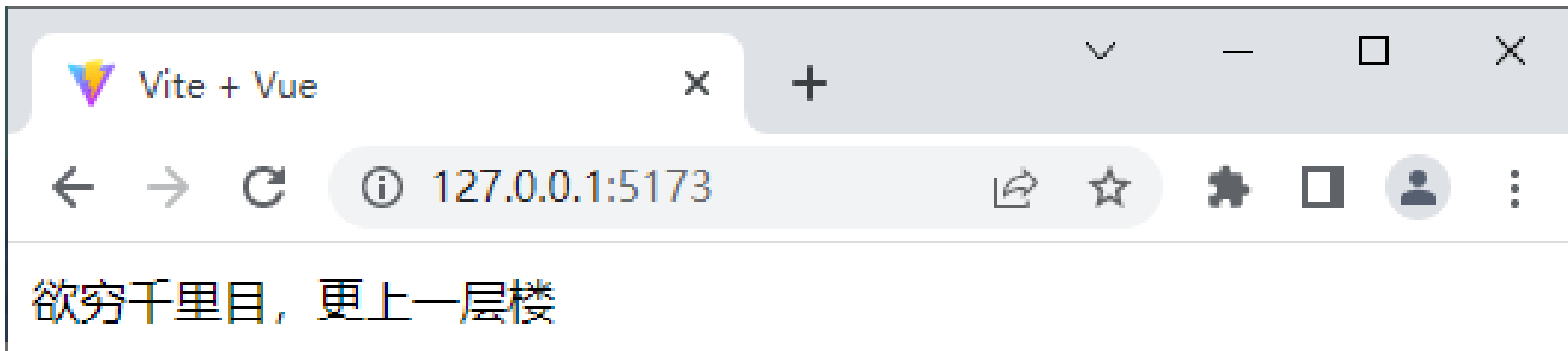
步骤2

2.2.2 响应式数据绑定

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，初始页面效果如下图所示。



等待2秒后的页面效果如下图所示。



2.2.2 响应式数据绑定

3. toRef()函数

一个待办事项的APP，里面有一个列表显示所有要完成的事情。列表中每一项待办事项都有一些信息：

- 任务内容：例如“明天上午 9 点开会”。
- 是否完成：一个勾选框，表示这个任务完成了没有。
- 优先级：可以是“高”、“中”、“低”。

在 Vue 里，所有的待办事项都放在一个“总的数据仓库”，就是一个 `reactive()` 对象。如果想修改某一个事项，点击该事项旁边的“编辑”按钮时，会弹出一个“编辑待办事项”的小窗口，这时候往往只需要针对某一个特定的待办事项进行修改。

2.2.2 响应式数据绑定

3. toRef()函数

toRef()函数用于将响应式对象中的单个属性转换为响应式数据。

使用toRef()函数定义响应式数据的语法格式如下。

```
响应式数据 = toRef(响应式对象, '属性名')
```



toRef()函数的第1个参数是响应式对象，第2个参数是待转换的属性名，返回值为转换后的响应式数据。

2.2.2 响应式数据绑定

演示toRef()函数的使用方法

步骤1

创建src\components\ToRef.vue文件。

步骤2

```
<template>
  <div>message的值: {{ message }}</div>
  <div>obj.message的值: {{ obj.message }}</div>
</template>
<script setup>
  import { reactive, toRef } from 'vue'
  const obj = reactive({ message: '黑发不知勤学早, 白首方悔读书迟' })
  const message = toRef(obj, 'message')
  setTimeout(() => {
    message.value = '少壮不努力, 老大徒伤悲'
  }, 2000)
</script>
```

2.2.2 响应式数据绑定

演示toRef()函数的使用方法

步骤1

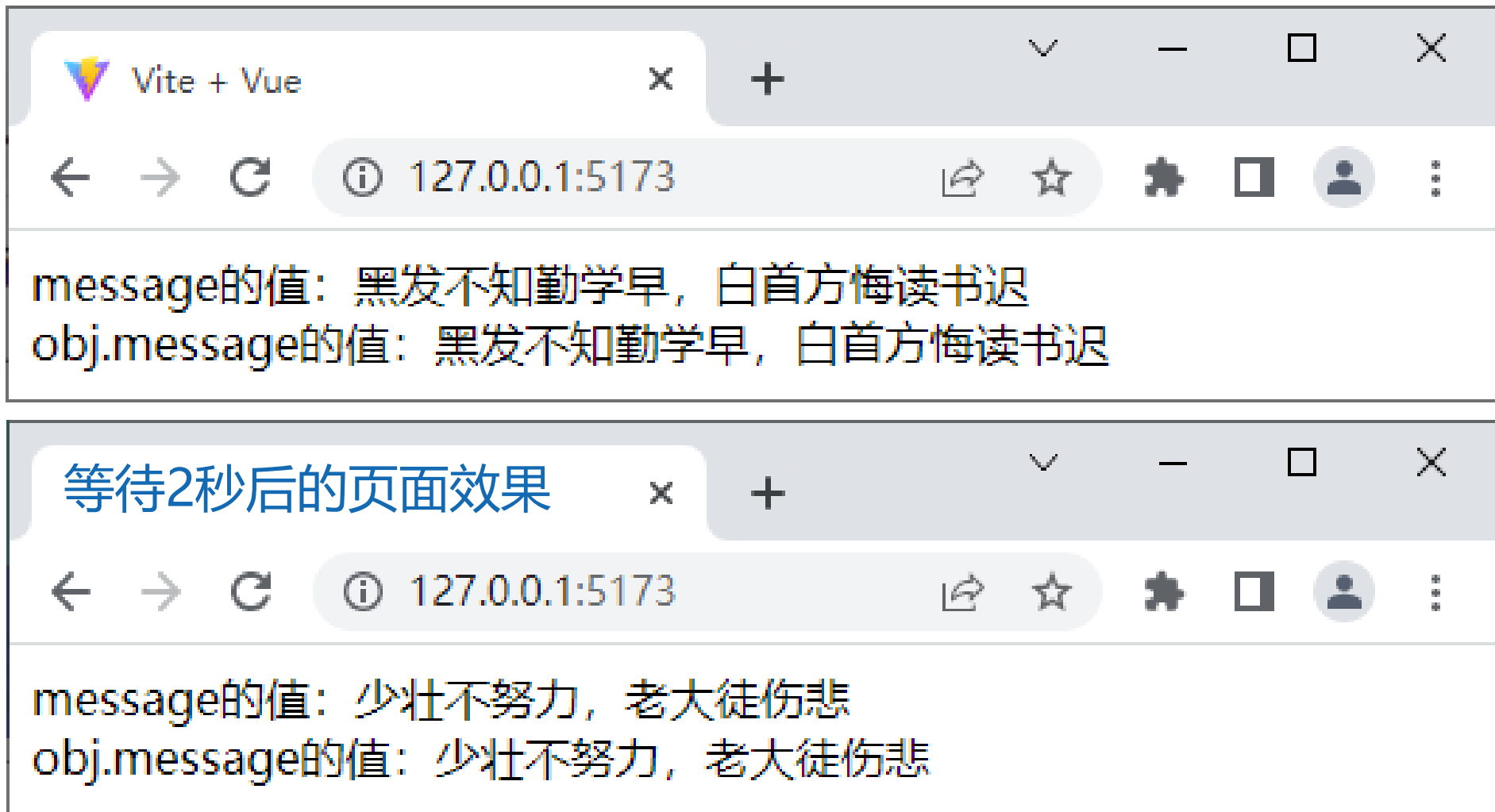
修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

```
import App from './components/ToRef.vue'
```

步骤2

2.2.2 响应式数据绑定

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，初始页面效果如下图所示。



2.2.2 响应式数据绑定

4. toRefs()函数

toRefs()函数用于将响应式对象中的所有属性转换为响应式数据。

使用toRefs()函数定义响应式数据的语法格式如下。

所有属性组成的对象 = toRefs(响应式对象)

2.2.2 响应式数据绑定

演示toRefs()函数的使用方法

步骤1

创建src\components\ToRefs.vue文件。

步骤2

```
<template>
  <div>message的值: {{ message }}</div>
  <div>obj.message的值: {{ obj.message }}</div>
</template>
<script setup>
import { reactive, toRefs } from 'vue'
const obj = reactive({ message: '盛年不重来，一日难再晨' })
let { message } = toRefs(obj)
setTimeout(() => {
  message.value = '及时当勉励，岁月不待人'
}, 2000)
</script>
```

2.2.2 响应式数据绑定

演示toRefs()函数的使用方法

步骤1

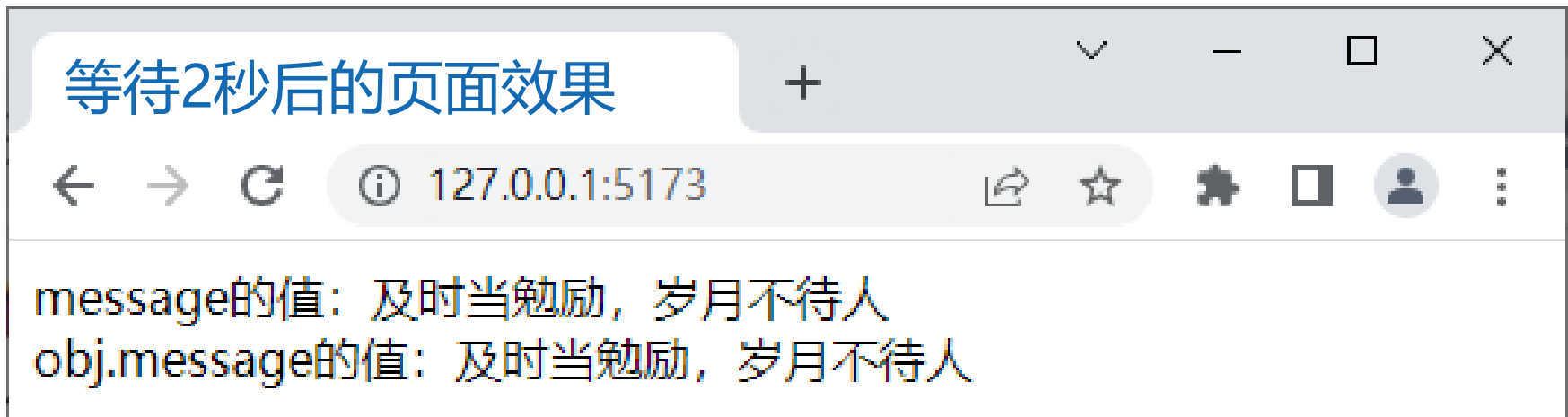
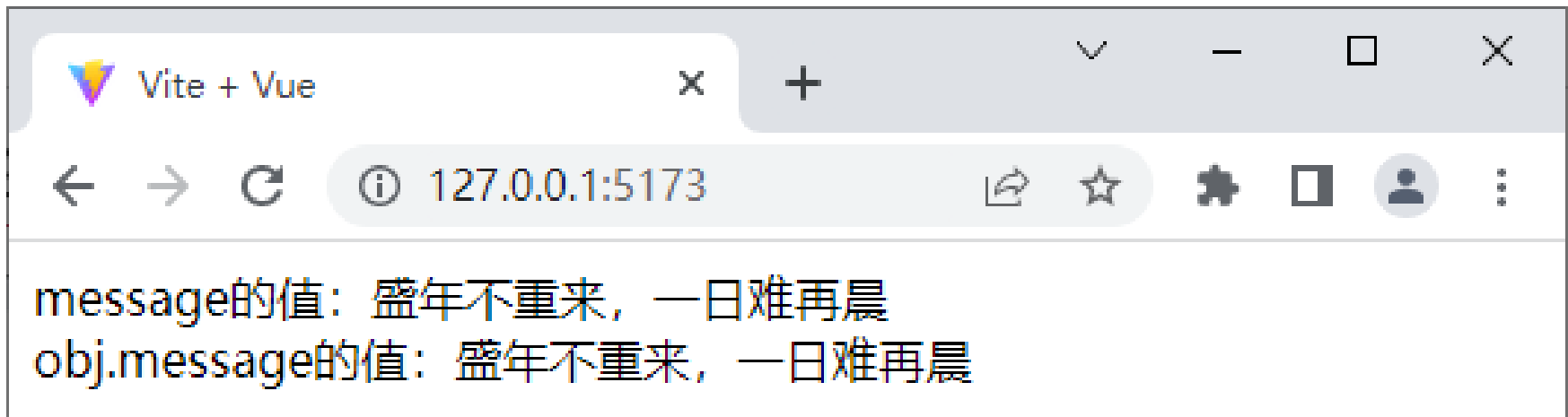
修改src\main.js文件，切换页面中显示的组件。

```
import App from './components/ToRefs.vue'
```

步骤2

2.2.2 响应式数据绑定

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，初始页面效果如下图所示。



2.2.2 响应式数据绑定 - 案例 todolist

```
<template>
  <div>
    <input v-model="newTask" placeholder="添加任务"
    @keyup.enter="addTask" />
    <ul>
      <li v-for="task in tasks" :key="task">{{ task }}</li>
    </ul>
  </div>
</template>
```

```
<script setup>
import { reactive, ref, toRefs } from 'vue'

const state = reactive({ tasks: ['学习 Vue', '写代码'] })
const { tasks } = toRefs(state)
const newTask = ref("")

const addTask = () => {
  if (newTask.value) {
    tasks.value.push(newTask.value)
    newTask.value = ""
  }
}
</script>
```

2.3

指令

2.3.1 内容渲染指令



掌握内容渲染指令，能够灵活运用v-text和v-html指令将字符串渲染到页面中

2.3.1 内容渲染指令



内容渲染指令用于渲染DOM元素的内容。

常见的内容渲染指令如下。

- v-text
- v-html

2.3.1 内容渲染指令

1. v-text

v-text用于渲染DOM元素的文本内容，如果文本内容中包含HTML标签，那么浏览器不会对其进行解析。v-text的语法格式如下。

```
<标签名 v-text="数据名"> </标签名>
```

2.3.1 内容渲染指令

演示v-text的使用方法

步骤1

创建src\components\VText.vue文件。

步骤2

```
<template>
  <div v-text="message"> </div>
</template>
<script setup>
const message = '<span>咬定青山不放松，立根原在破岩中</span>'
</script>
```

2.3.1 内容渲染指令

演示v-text的使用方法

步骤1

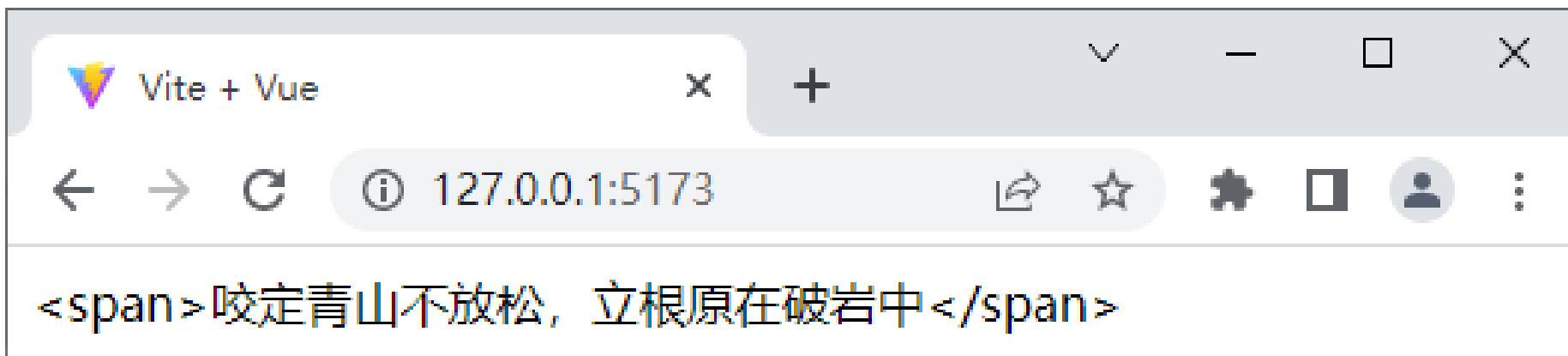
修改src\main.js文件，**切换**页面中**显示**的**组件**。

```
import App from './components/VText.vue'
```

步骤2

2.3.1 内容渲染指令

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，`v-text`的运行结果如下图所示。



2.3.1 内容渲染指令

2. v-html

在使用Vue进行内容渲染时，如果内容中包含HTML标签并且希望这些标签被浏览器解析，则可以使用v-html。**v-html**用于**渲染DOM元素的HTML内容**，其用法与v-text相似。v-html的语法格式如下。

```
<标签名 v-html="数据名"> </标签名>
```

2.3.1 内容渲染指令

演示v-html的使用方法

步骤1

创建src\components\VHtml.vue文件。

步骤2

```
<template>
  <div v-html="html"> </div>
</template>
<script setup>
const html = '<strong>千磨万击还坚劲，任尔东西南北风</strong>'
</script>
```

2.3.1 内容渲染指令

演示v-html的使用方法

步骤1

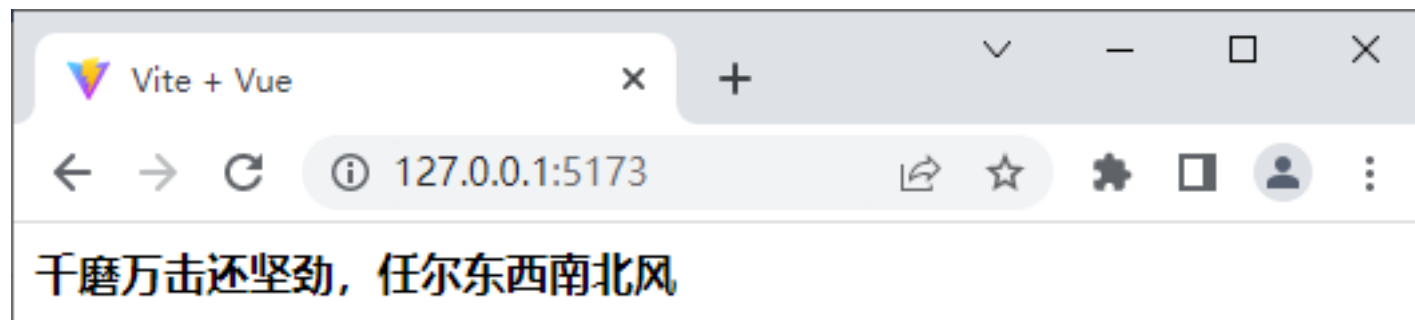
修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

```
import App from './components/VHtml.vue'
```

步骤2

2.3.1 内容渲染指令

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，**v-html**的运行结果如下图所示。



2.3.2 属性绑定指令



掌握属性绑定指令，能够灵活运用v-bind指令给目标元素的属性动态绑定值

2.3.2 属性绑定指令

在Vue开发中，有时需要绑定DOM元素的属性，从而更好地控制属性的值，此时可以使用属性绑定指令v-bind来实现。v-bind的语法格式如下。

```
<标签名 v-bind:属性名="数据名"> </标签名>
```

v-bind还支持将属性与字符串拼接表达式绑定，示例代码如下。

```
<div :id="'list' + index"> </div>
```

```

```

```

```

2.3.2 属性绑定指令

演示v-bind的使用方法

步骤1

创建src\components\VBind.vue文件。

步骤2

```
<template>
  <p> <input type="text" v-bind:placeholder="username"> </p>
  <p> <input type="password" :placeholder="password"> </p>
</template>
<script setup>
const username = '请输入用户名'
const password = '请输入密码'
</script>
```


2.3.2 属性绑定指令

演示v-bind的使用方法

步骤1

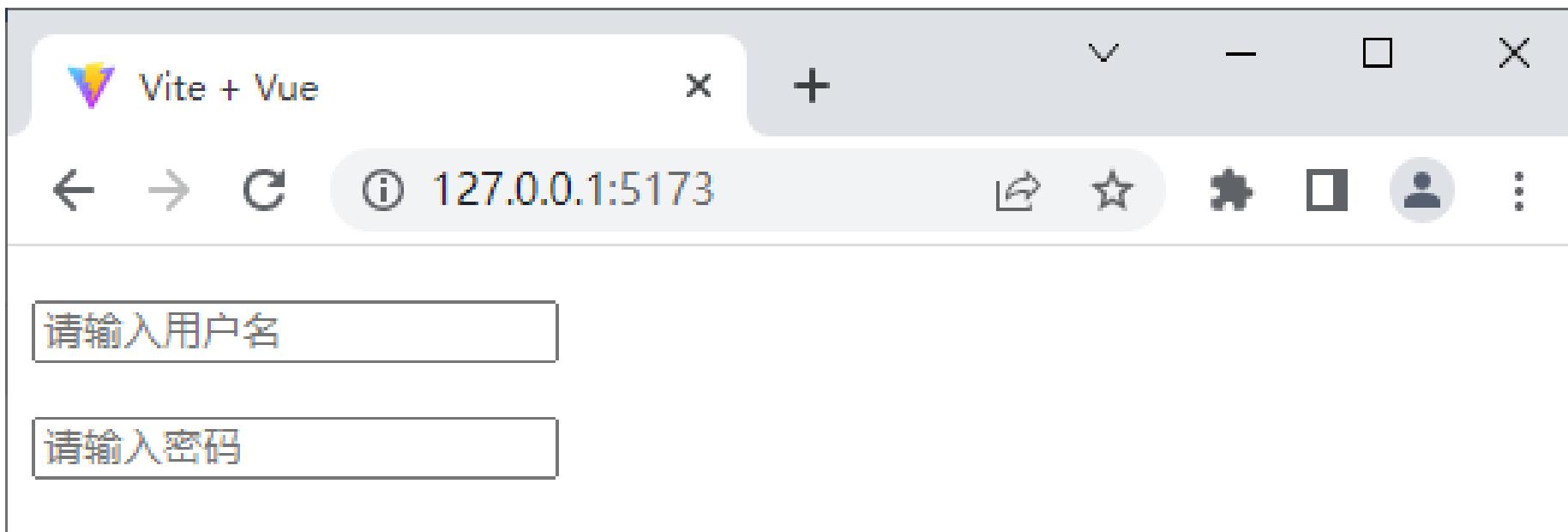
修改src\main.js文件，**切换**页面中**显示**的**组件**。

```
import App from './components/VBind.vue'
```

步骤2

2.3.2 属性绑定指令

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，[v-bind的运行结果](#)如下图所示。



V-bind是单向数据绑定。
当改变数据名时，属性名会自动更新；
而当属性名改变时，数据名的值不会同步变化。

2.3.2 属性绑定指令

- 单向数据绑定 (One-Way Binding) : 数据只能从 JavaScript (Vue 实例) 传递到 HTML (视图), 但不能反向修改。
- 双向数据绑定 (Two-Way Binding) : 数据在 Vue 实例和视图之间同步, 视图的修改会自动更新数据, 数据的修改也会更新视图。

2.3.2 属性绑定指令

```
<template>
  <h1>{{ title }}</h1>
  
  <p :style="{ color: textColor }">这是一段彩色文本</p>
</template>

<script setup>
import { ref } from 'vue'
const title = ref("Vue 单向绑定示例")
const imageUrl = ref("https://example.com/image.jpg")
const textColor = ref("blue")
</script>
```

2.3.3 事件绑定指令



- 掌握事件绑定指令，能够灵活运用v-on指令给目标元素绑定事件

2.3.3 事件绑定指令

在Vue开发中，有时需要给DOM元素绑定事件，从而利用事件实现交互效果，这时可以利用事件绑定指令v-on来实现。v-on的语法格式如下。

```
<标签名 v-on:事件名="事件处理器"> </标签名>
```

在上述语法格式中，事件名即DOM中的事件名，例如click、input、keyup等；事件处理器可以是方法名或内联JavaScript语句。

- 如果逻辑复杂，事件处理器建议使用方法名，方法需要在<script>标签中定义；
- 如果逻辑简单，只有一行代码，则可以使用内联JavaScript语句。

另外，v-on还有简写形式，可以将“v-on:事件名”简写为“@事件名”。

```
<button v-on:click="handleClick">点我</button>
```

```
@click="count++"
```

2.3.3 事件绑定指令

演示v-on的使用方法

步骤1

创建src\components\VOn.vue文件。

步骤2

```
<template>
  <button @click="showInfo">输出信息</button>
</template>
<script setup>
const showInfo = () => {
  console.log('欢迎来到Vue.js的世界! ')
}
</script>
```

2.3.3 事件绑定指令

演示v-on的使用方法

步骤1

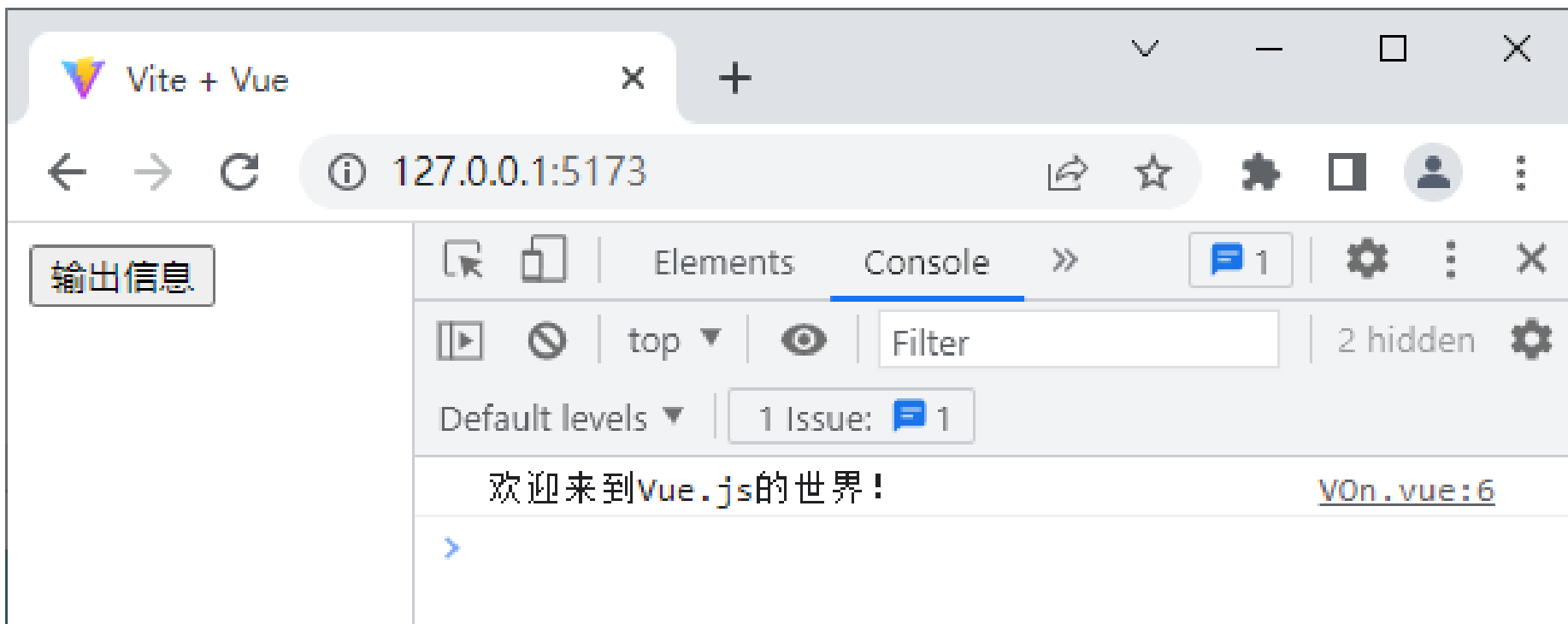
修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

```
import App from './components/VOn.vue'
```

步骤2

2.3.3 事件绑定指令

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，单击“输出信息”按钮后的页面效果和控制台如下图所示。



2.3.4 双向数据绑定指令



掌握双向数据绑定指令，能够灵活运用
`v-model`指令在表单元素上实现数据的
双向绑定

2.3.4 双向数据绑定指令

Vue为开发者提供了v-model指令来实现双向数据绑定，使用它可以在input、textarea和select元素上创建双向数据绑定，它会根据使用的元素自动选取对应的属性和事件组合，负责监听用户的输入事件并更新数据。v-model的语法格式如下。

```
<标签名 v-model="数据名"></标签名>
```

2.3.4 双向数据绑定指令

v-model内部会为不同的元素绑定不同的属性和事件，具体如下。

- **textarea**元素 + **text**类型的**input**元素会绑定 → **value**属性和**input**事件。
- **checkbox**类型的**input**元素 + **radio**类型的**input**元素会绑定 → **checked**属性和**change**事件。
- **select**元素会绑定 → **value**属性和**change**事件。

例如，使用v-model实现输入框的值与变量保持同步，示例代码如下。

```
<input v-model="info">
```

2.3.4 双向数据绑定指令

- 需要注意的是，v-model会忽略掉所有表单元素的value，checked，selected属性的初始值，而将v-model的数据作为数据来源。
- 另外，为了方便对用户输入的内容进行处理，v-model提供了3个修饰符。v-model的修饰符如下表所示。

修饰符	作用
.number	自动将用户输入的值转换为数字类型
.trim	自动过滤用户输入的首尾空白字符
.lazy	在change事件而非input事件触发时更新数据

2.3.4 双向数据绑定指令

演示v-model的使用方法

步骤1

创建src\components\VModel.vue文件。

步骤2

```
<template>
  请输入姓名: <input type="text" v-model="username">
  <div>姓名是: {{ username }}</div>
</template>
<script setup>
import { ref } from 'vue'
const username = ref('zhangsan')
</script>
```

2.3.4 双向数据绑定指令

演示v-model的使用方法

步骤1

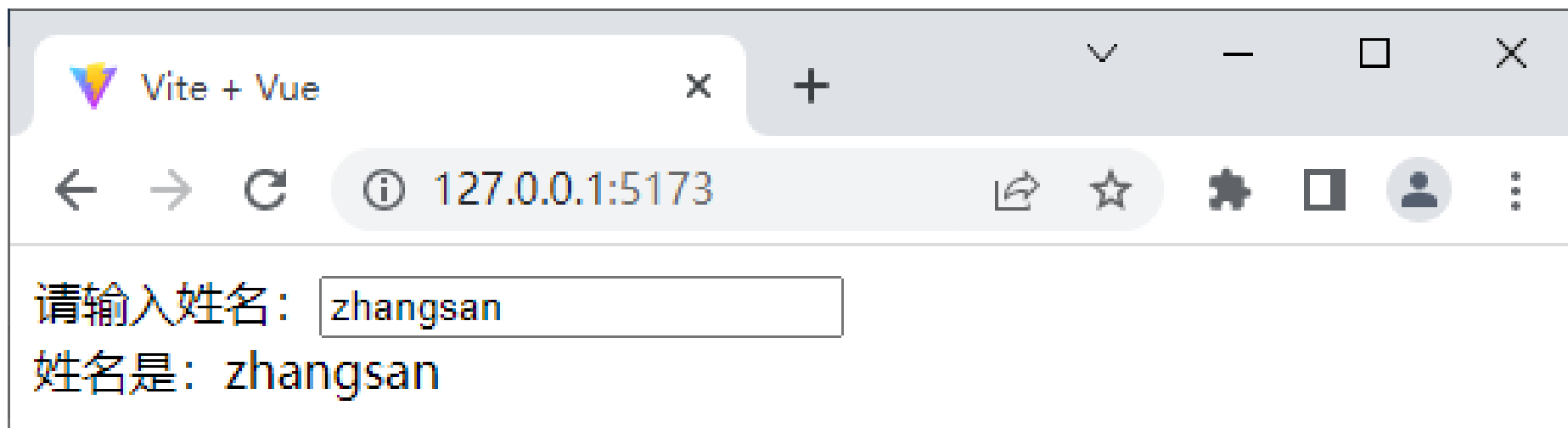
修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

```
import App from './components/VModel.vue'
```

步骤2

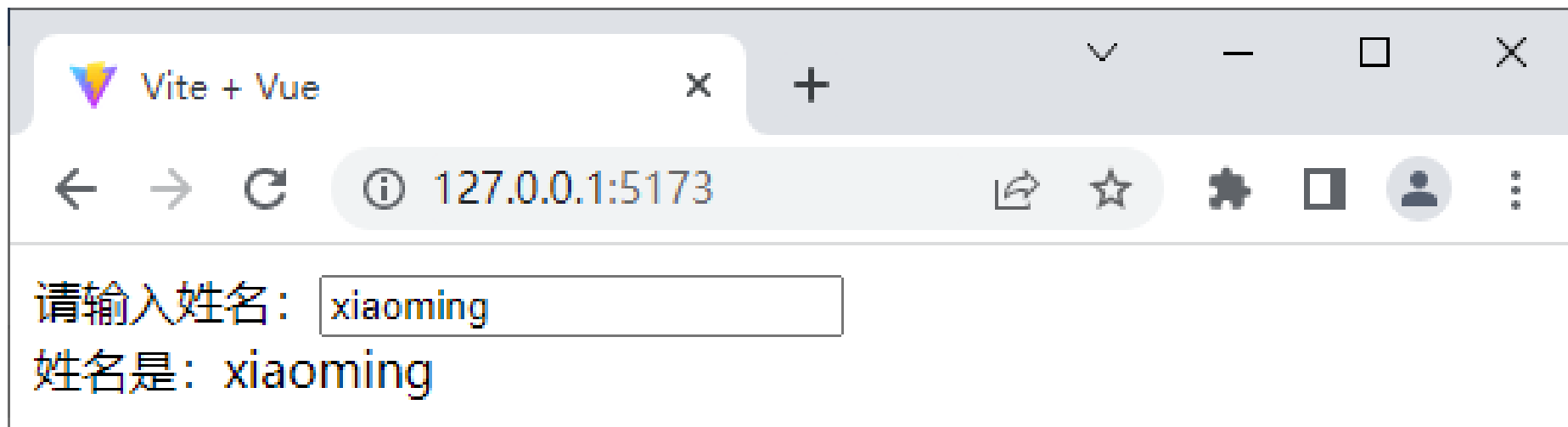
2.3.4 双向数据绑定指令

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，`v-model`的初始页面效果如下图所示。



2.3.4 双向数据绑定指令

在输入框中输入“xiaoming”，页面效果如下图所示。



2.3.4 双向数据绑定指令

演示.number修饰符的使用方法

步骤1

在src\components\VModel.vue文件的<script>标签中添加代码，定义响应式数据n1和n2。

```
const n1 = ref(1)
const n2 = ref(2)
```

步骤2

步骤3

2.3.4 双向数据绑定指令

演示.number修饰符的使用方法

步骤1

在src\components\VModel.vue文件的<template>标签中添加代码，通过v-model实现双向数据绑定。

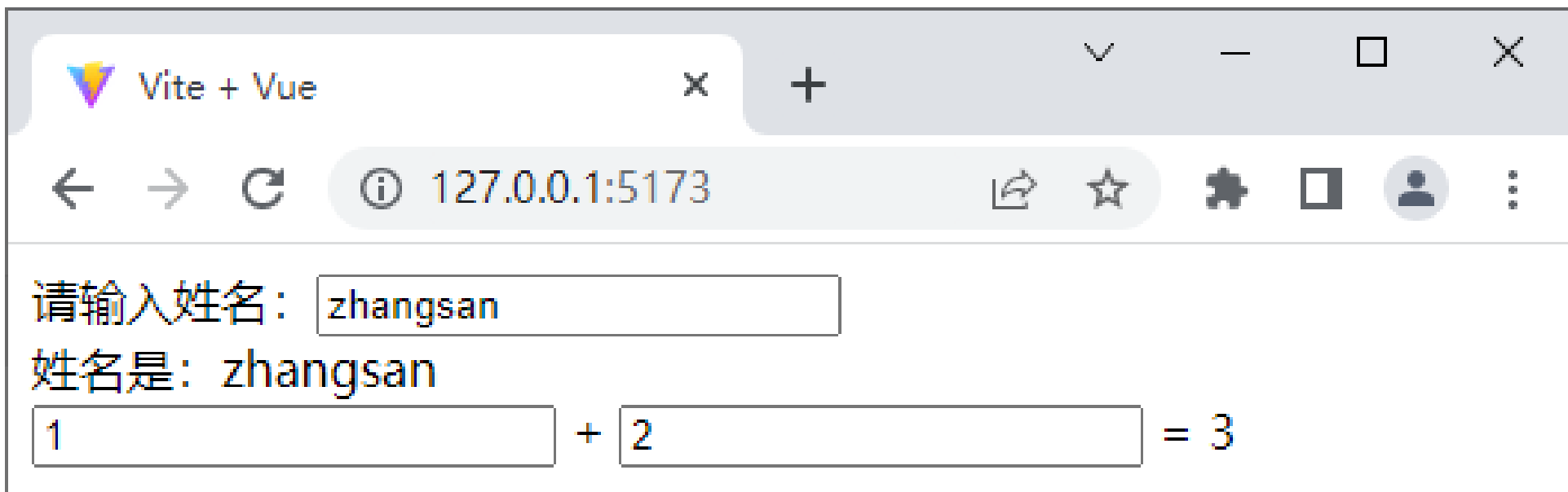
步骤2

```
<input type="text" v-model="n1"> + <input  
type="text" v-model="n2">  
= {{ n1 + n2 }}
```

步骤3

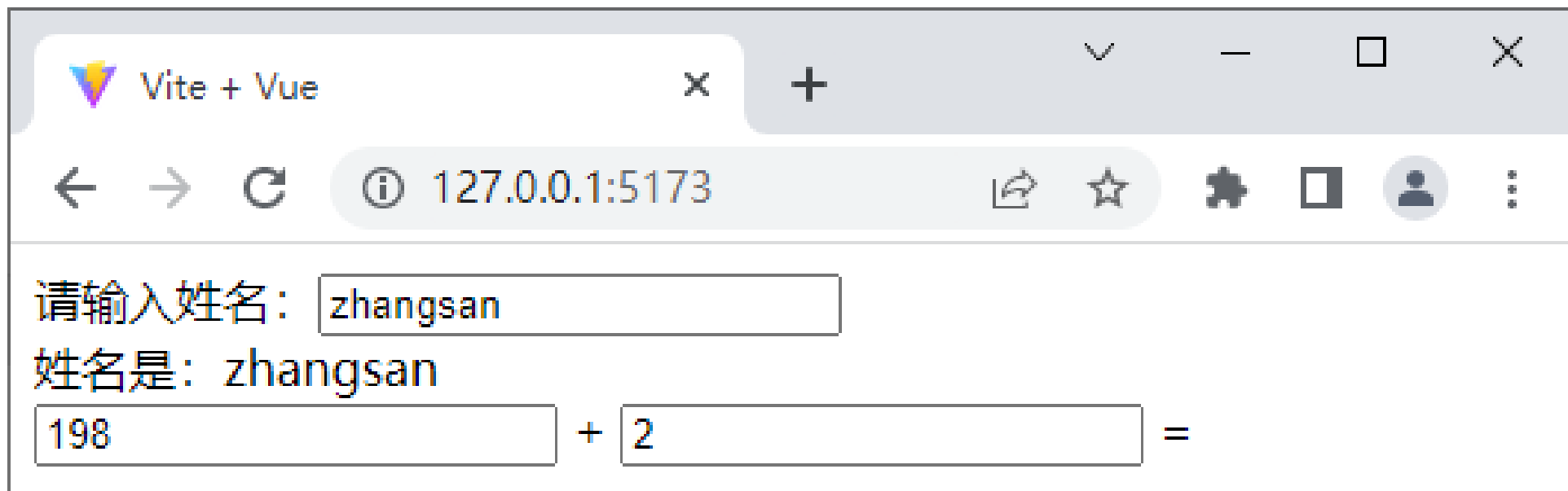
2.3.4 双向数据绑定指令

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，未使用.number修饰符的初始页面效果如下图所示。



2.3.4 双向数据绑定指令

改变n1的值为“198”，查看n1 + n2的值。改变n1的值为“198”的页面效果如下图所示。



2.3.4 双向数据绑定指令

演示.number修饰符的使用方法

步骤1

修改输入框的代码，通过添加.number修饰符实现将用户输入的数字自动转换成数字类型。

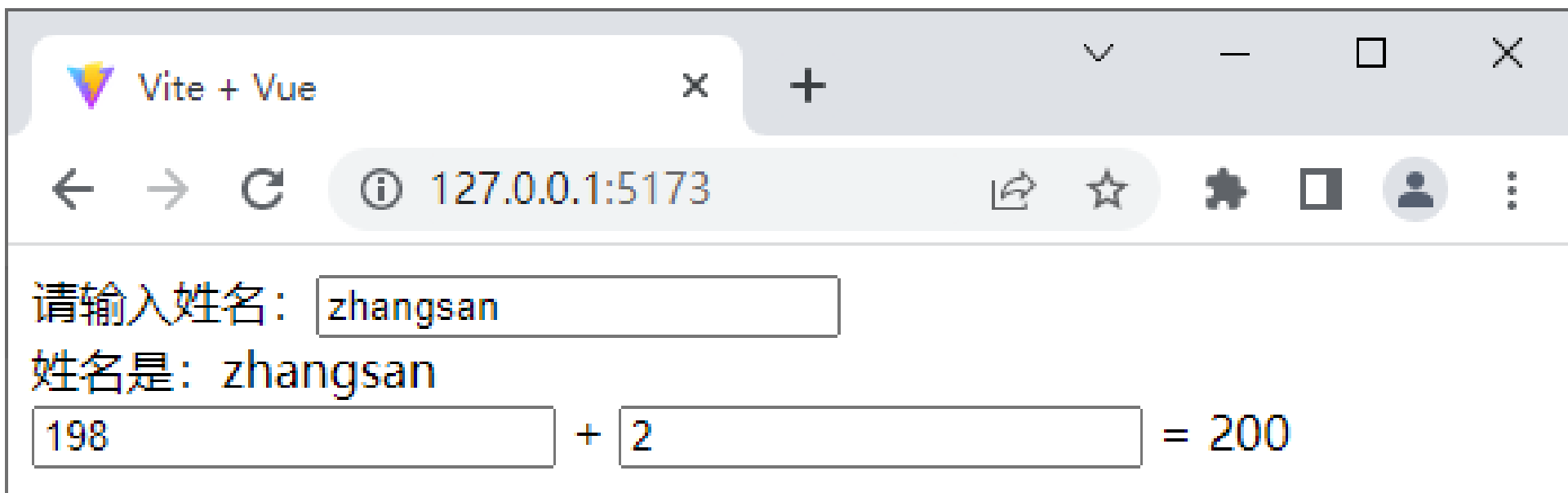
步骤2

```
<input type="text" v-model.number="n1"> +  
<input type="text" v-model.number="n2">
```

步骤3

2.3.4 双向数据绑定指令

改变n1的值为“198”，查看n1 + n2的值。使用.number修饰符后改变n1的值为“198”的页面效果如下图所示。

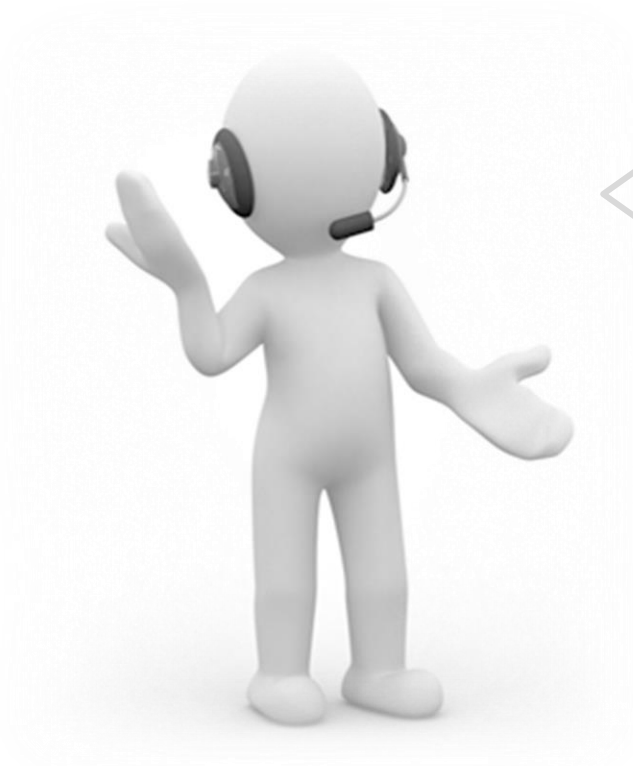


2.3.5 条件渲染指令



掌握条件渲染指令，能够灵活运用v-if、v-show指令根据不同的条件渲染不同的标签

2.3.5 条件渲染指令



在Vue中，当需要根据不同的判断结果显示不同的DOM元素时，可以通过条件渲染指令来实现。条件渲染指令可以辅助开发者按需控制DOM元素的显示与隐藏。

条件渲染指令如下。

- v-if
- v-show

2.3.5 条件渲染指令

1. v-if

v-if是根据布尔值切换元素的显示或隐藏状态，本质是通过操作DOM元素来切换显示状态。

- 当给定的值为**true**时，元素存在于DOM树中；
- 当给定的值为**false**时，元素从DOM树中移除。

2.3.5 条件渲染指令

v-if有两种使用方式。



直接给定一个条件，控制单个元素的显示或隐藏

```
<标签名 v-if="条件"> </标签名>
```

通过v-if结合v-else-if、v-else来控制不同元素的显示或隐藏

```
<标签名 v-if="条件A">展示A</标签名>  
<标签名 v-else-if="条件B">展示B</标签名>  
<标签名 v-else>展示C</标签名>
```

2.3.5 条件渲染指令

演示v-if的使用方法

步骤1

创建src\components\VIf.vue文件。

步骤2

```
<template>
  小明的学习评定等级为:
  <p v-if="type === 'A'">优秀</p>
  <p v-else-if="type === 'B'">良好</p>
  <p v-else>差</p>
  <button @click="type = 'A'">切换成优秀</button>
  <button @click="type = 'B'">切换成良好</button>
  <button @click="type = 'C'">切换成差</button>
</template>
<script setup>import { ref } from 'vue'
const type = ref('B')
</script>
```

2.3.5 条件渲染指令

演示v-if的使用方法

步骤1

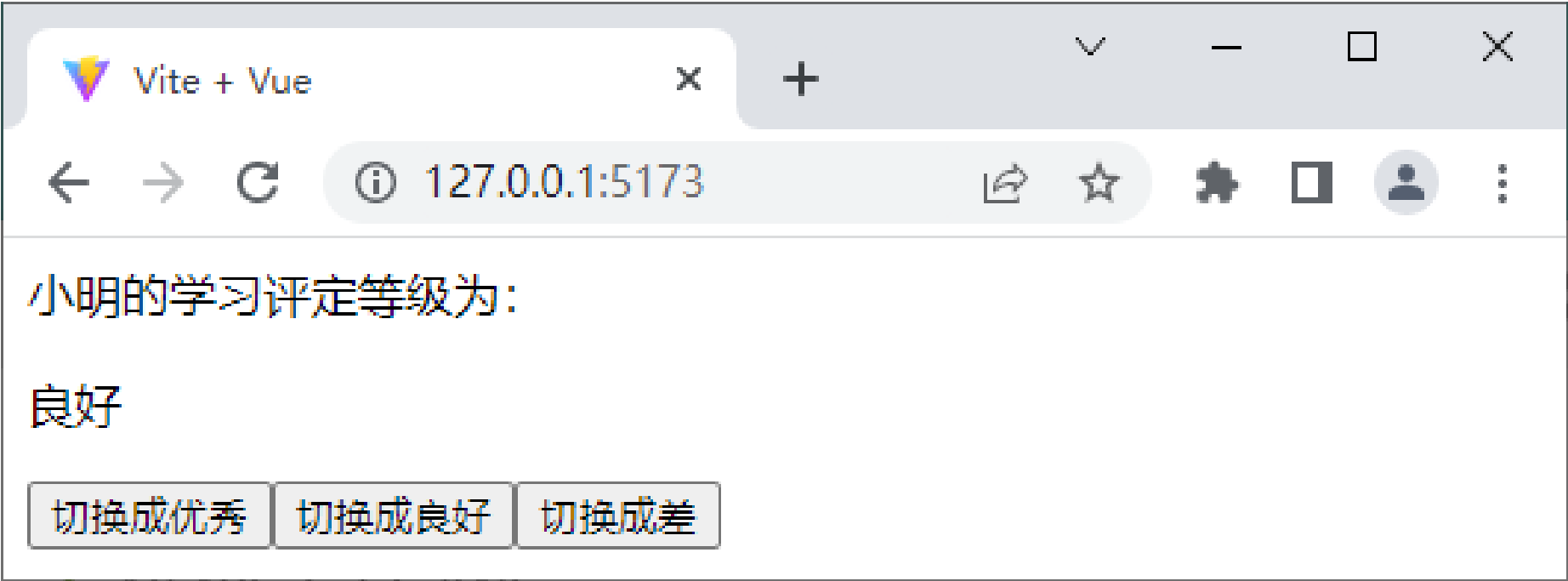
修改src\main.js文件，**切换**页面中**显示**的**组件**。

```
import App from './components/VIf.vue'
```

步骤2

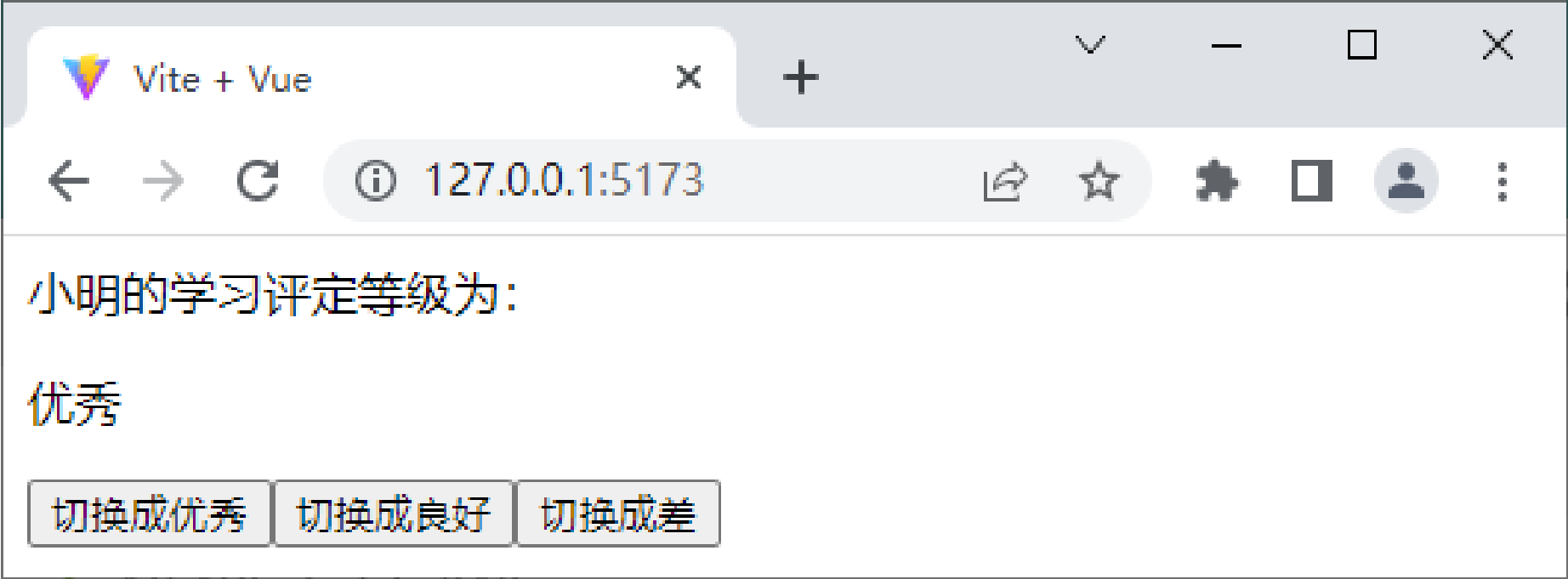
2.3.5 条件渲染指令

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，v-if的初始页面效果如下图所示。



2.3.5 条件渲染指令

单击“切换成优秀”按钮后的页面效果如下图所示。



2.3.5 条件渲染指令

2. v-show

`v-show`与`v-if`都用来决定某一个元素是否在页面上显示出来。

`v-show`的原理是通过为元素添加或移除`display: none`样式来实现元素的显示或隐藏。

当需要频繁切换某个元素的显示或隐藏时，使用`v-show`会更节省性能开销；而当只需要切换一次显示或隐藏时，使用`v-if`更合理。

2.3.5 条件渲染指令

演示v-show的使用方法

步骤1

创建src\components\VShow.vue文件。

步骤2

```
<template>
  <p v-if="flag">通过v-if控制的元素</p>
  <p v-show="flag">通过v-show控制的元素</p>
  <button @click="flag = !flag">显示/隐藏</button>
</template>
<script setup>
import { ref } from 'vue'
const flag = ref(true)
</script>
```

2.3.5 条件渲染指令

演示v-show的使用方法

步骤1

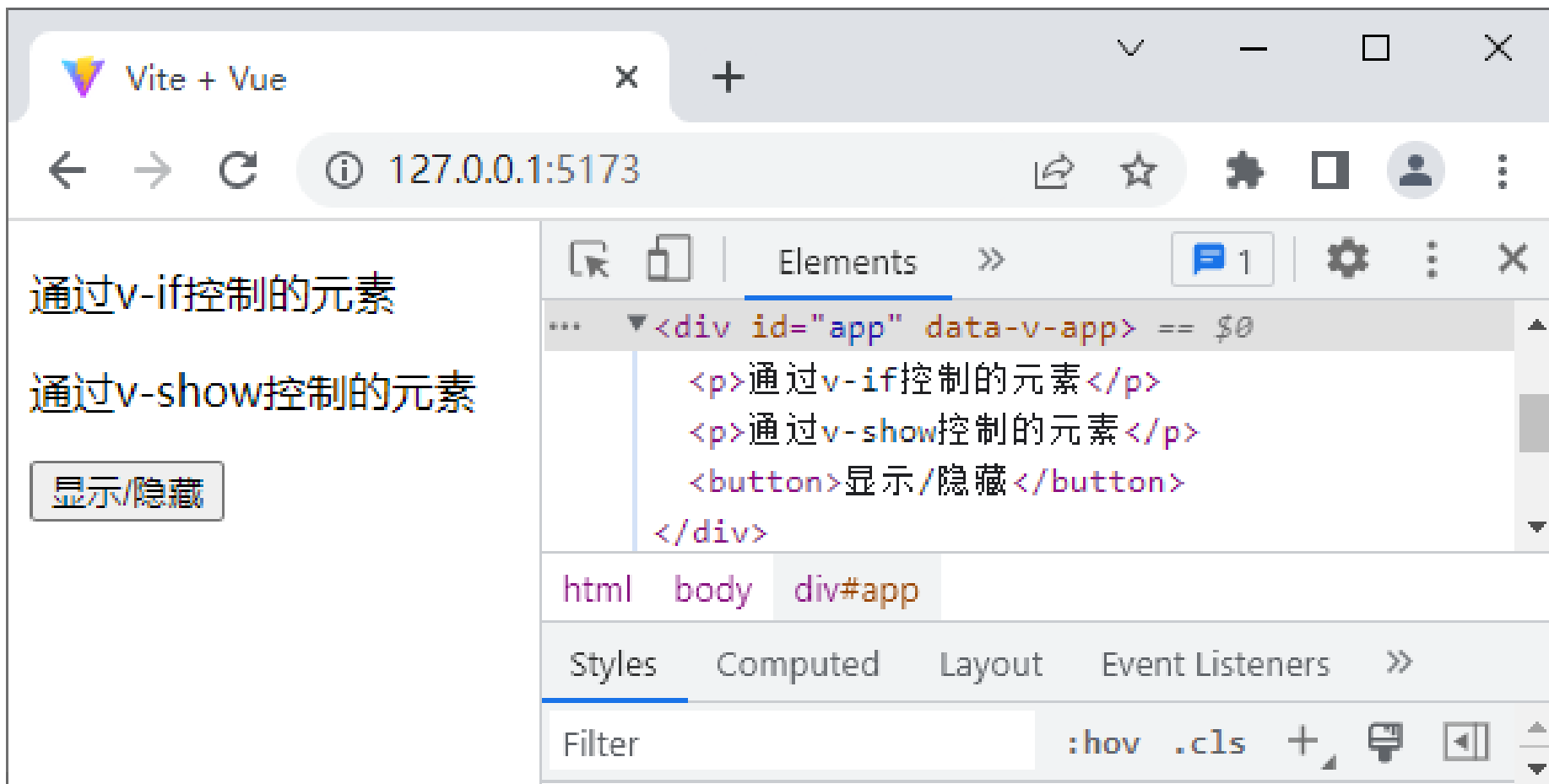
修改src\main.js文件，**切换**页面中**显示**的**组件**。

```
import App from './components/VShow.vue'
```

步骤2

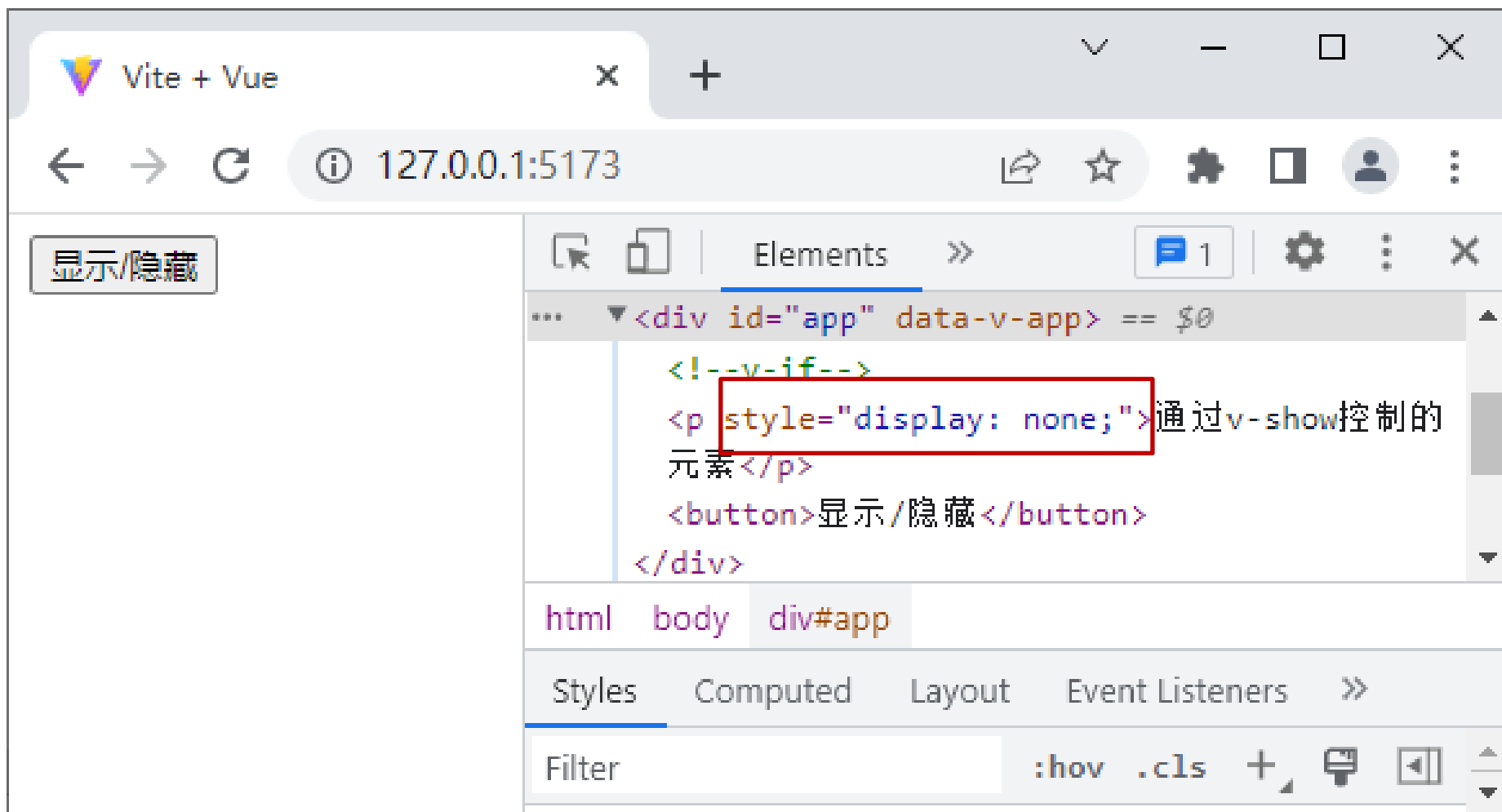
2.3.5 条件渲染指令

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，`v-show`的初始页面效果如下图所示。



2.3.5 条件渲染指令

单击“显示/隐藏”按钮后的页面效果如下图所示。



2.3.6 列表渲染指令



掌握列表渲染指令，能够灵活运用v-for指令将数组、对象等中的数据渲染到页面中

2.3.6 列表渲染指令



- 在开发购物应用时，为了方便用户查找商品信息，通常会以商品列表的形式展示商品信息。
- 在商品列表中，每件商品的结构都是相同的，如果每件商品的结构都要手动定义，会非常麻烦。
- 为此，Vue提供了列表渲染指令v-for。开发者只需在模板中定义一件商品的结构，v-for便会根据提供的数据自动渲染商品列表中所有的商品。

2.3.6 列表渲染指令

使用
v-for
循环
渲染
列表

根据数组渲染列表



<标签名 v-for="(item, index) in arr"> </标签名>

根据对象渲染列表



<标签名 v-for="(item, name, index) in obj"> </标签名>

根据数字渲染列表



<标签名 v-for="(item, index) in num"> </标签名>

根据字符串渲染列表



<标签名 v-for="(item, index) in str"> </标签名>

2.3.6 列表渲染指令

使用v-for（根据list数组中的元素）渲染列表后，当在list数组中删除一个元素后，index会发生变化，v-for会重新渲染列表，导致性能下降。

为了给v-for一个提示，以便它能跟踪每个节点的身份，从而对现有元素进行重用和重新排序，建议通过key属性为列表中的每一项提供具有唯一性的值，示例代码如下。

```
<div v-for="item in items" :key="item.id"> </div>
```


2.3.6 列表渲染指令

演示使用v-for根据一维数组渲染列表

步骤1

创建src\components\VFor1.vue文件。

步骤2

```
<template>
  <div v-for="(item, index) in list" :key="index">
    索引是: {{ index }} --- 元素的内容是: {{ item }}
  </div>
</template>
<script setup>
import { reactive } from 'vue'
const list = reactive(['HTML', 'CSS', 'JavaScript'])
</script>
```

2.3.6 列表渲染指令

演示使用v-for根据一维数组渲染列表

步骤1

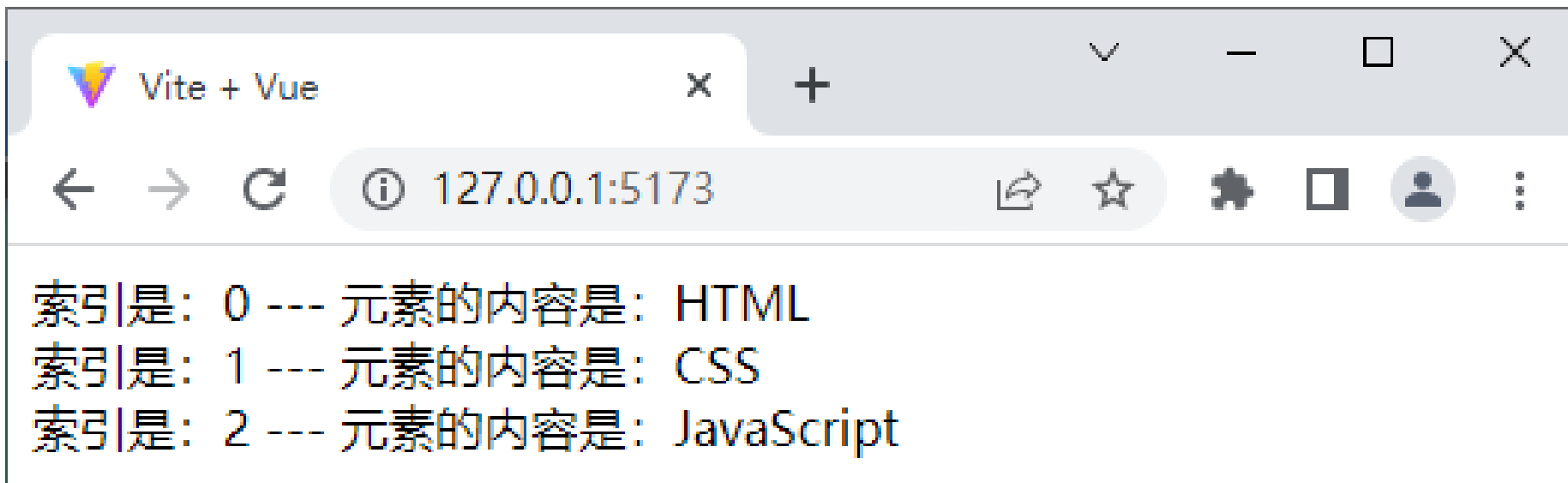
修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

```
import App from './components/VFor1.vue'
```

步骤2

2.3.6 列表渲染指令

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，根据一维数组渲染列表的页面效果如下图所示。



2.3.6 列表渲染指令

演示使用v-for根据对象数组渲染列表

步骤1

创建src\components\VFor2.vue文件。

步骤2

```
<template>
  <div v-for="item in list" :key="item.id">
    id是: {{ item.id }} --- 元素的内容是: {{ item.message }}
  </div>
</template>
<script setup>
import { reactive } from 'vue'
const list = reactive([
  { id: 1, message: '梅', }, { id: 2, message: '兰', },
  { id: 3, message: '竹', }, { id: 4, message: '菊', }
])
</script>
```

2.3.6 列表渲染指令

演示使用v-for根据对象数组渲染列表

步骤1

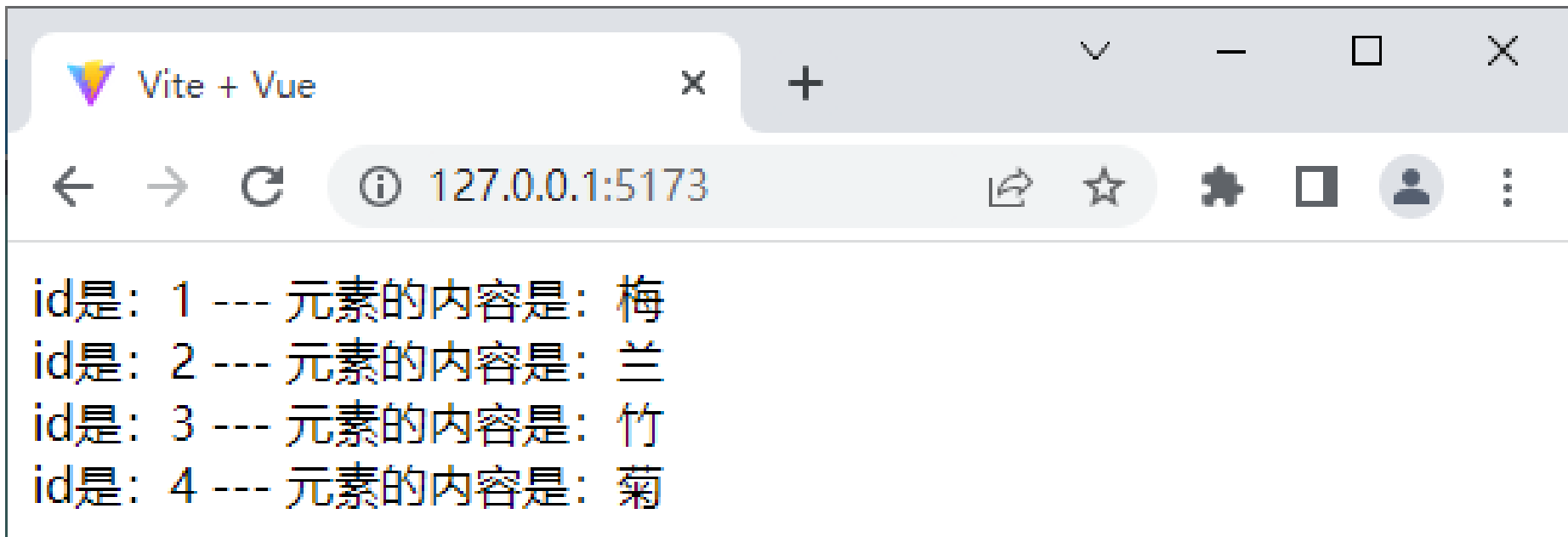
修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

```
import App from './components/VFor2.vue'
```

步骤2

2.3.6 列表渲染指令

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，根据对象数组渲染列表的页面效果如下图所示。



2.3.6 列表渲染指令

演示使用v-for根据对象渲染列表

步骤1

创建src\components\VFor3.vue文件。

步骤2

```
<template>
  <div v-for="(value, name) in user" :key="name">
    属性名是: {{ name }} --- 属性值是: {{ value }}
  </div>
</template>
<script setup>
import { reactive } from 'vue'
const user = reactive({ id: 11, name: '小明', gender: '男' })
</script>
```

2.3.6 列表渲染指令

演示使用v-for根据对象渲染列表

步骤1

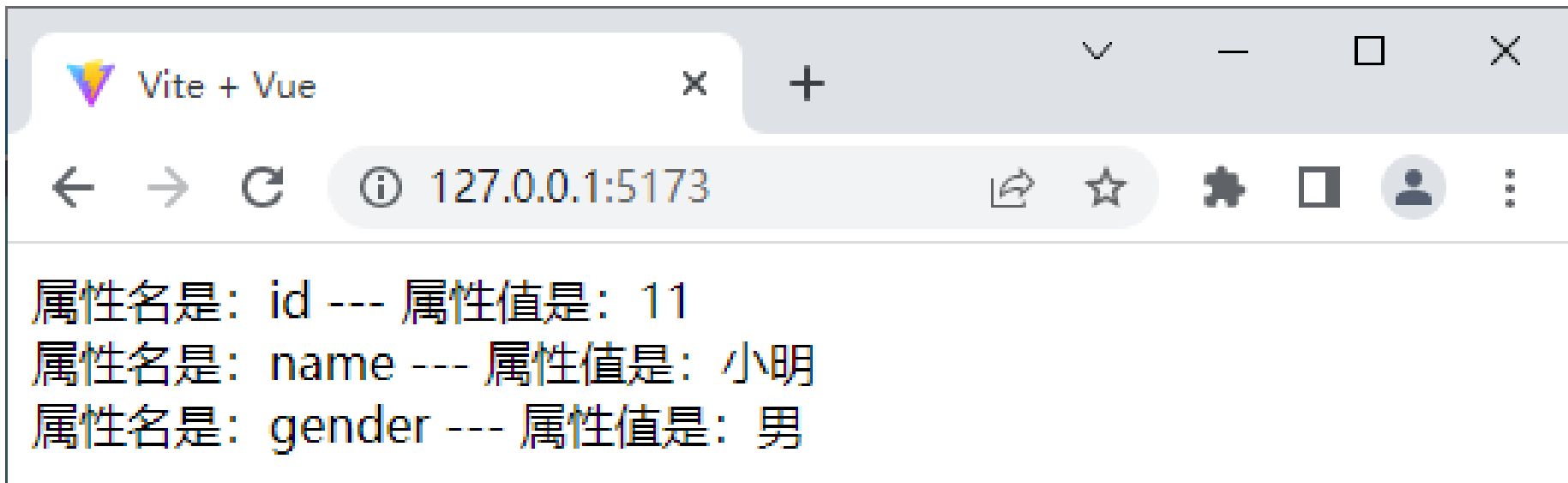
修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

```
import App from './components/VFor3.vue'
```

步骤2

2.3.6 列表渲染指令

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，根据对象渲染列表的页面效果如下图所示。



2.4

事件对象

2.4 事件对象



掌握事件对象，能够灵活运用事件对象
获取和修改DOM元素的属性

2.4 事件对象

什么是事件对象？



2.4 事件对象



事件对象是在事件触发时产生的对象，该对象保存了事件触发时的相关信息。

事件对象有以下两种获取方式。

- 通过事件方法的参数获取事件对象
- 通过\$event获取事件对象

2.4 事件对象

1. 通过事件方法的参数获取事件对象

在v-on绑定的用于处理事件的方法中，可以接收到一个参数，这个参数就是事件对象，示例代码如下。

```
<template>
  <button @click="greet">Greet</button>
</template>
<script setup>
const greet = event => console.log(event)
</script>
```

2.4 事件对象

2. 通过\$event获取事件对象

`$event`是Vue提供的内置变量，使用它可以获取事件对象，示例代码如下。

```
<template>
  <button @click="change($event)">change</button>
</template>
<script setup>
const change = event => console.log(event)
</script>
```

2.4 事件对象

演示事件对象的使用方法

步骤1

创建src\components\EventObject.vue文件。

步骤2

```
<template>
  <div>count的值为: {{ count }}</div>
  <button @click="addCount">count+1 </button>
</template>
<script setup>
import { ref } from 'vue'
const count = ref(1)
const addCount = event => {
  count.value++
  if (count.value % 2 === 0) { event.target.style.border = '3px
dotted' } else { event.target.style.border = '3px solid' } }
</script>
```


2.4 事件对象

演示事件对象的使用方法

步骤1

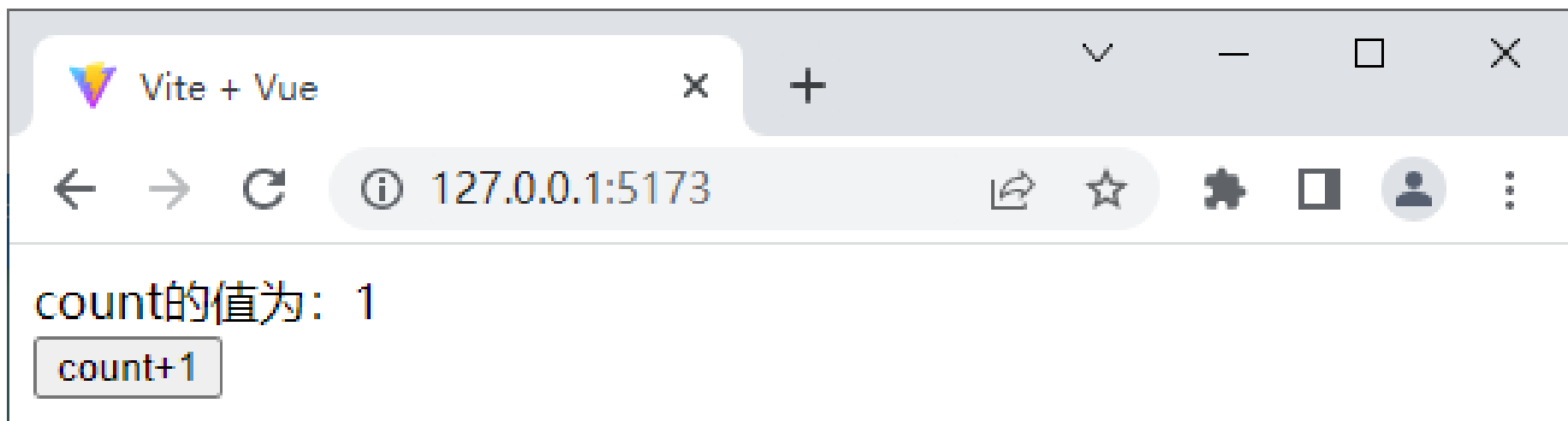
修改src\main.js文件，切换页面中显示的组件。

```
import App from './components/EventObject.vue'
```

步骤2

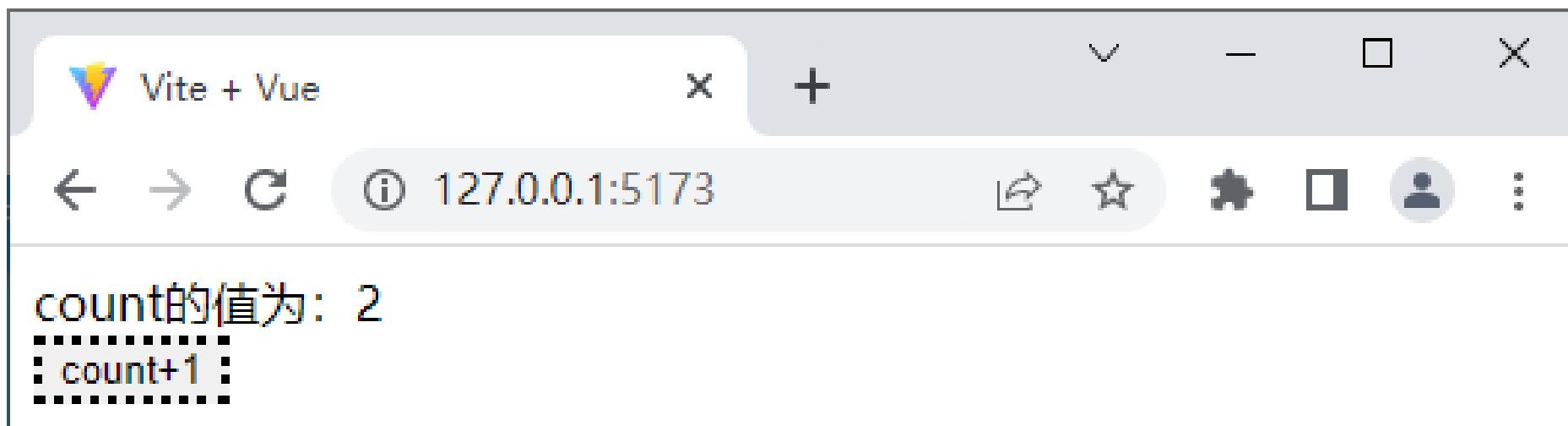
2.4 事件对象

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，通过事件方法的参数获取事件对象的页面效果如下图所示。



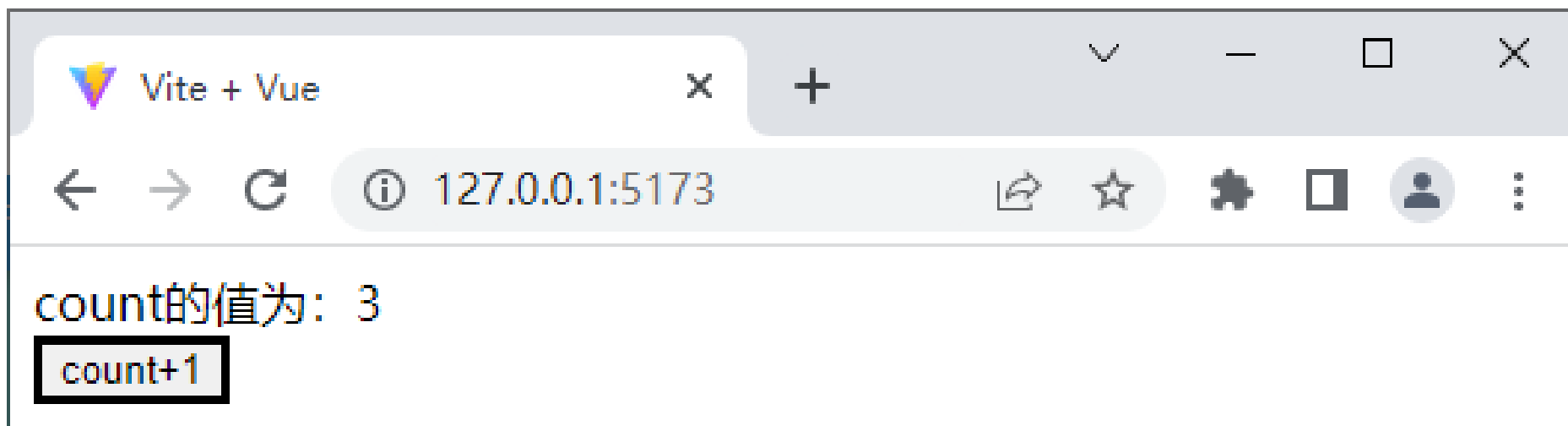
2.4 事件对象

单击 “count+1” 按钮后的页面效果如下图所示。



2.4 事件对象

再次单击 “count+1” 按钮后的页面效果如下图所示。



2.4 事件对象

演示通过\$event获取事件对象的方式

步骤1

修改src\components\EventObject.vue文件，编写addCountN()方法。

步骤2

```
const addCountN = (n, event) => {  
  count.value += n  
  if (count.value % 2 === 0) {  
    event.target.style.border = '3px dotted'  
  } else {  
    event.target.style.border = '3px solid'  
  }  
}
```

2.4 事件对象

演示通过\$event获取事件对象的方式

步骤1

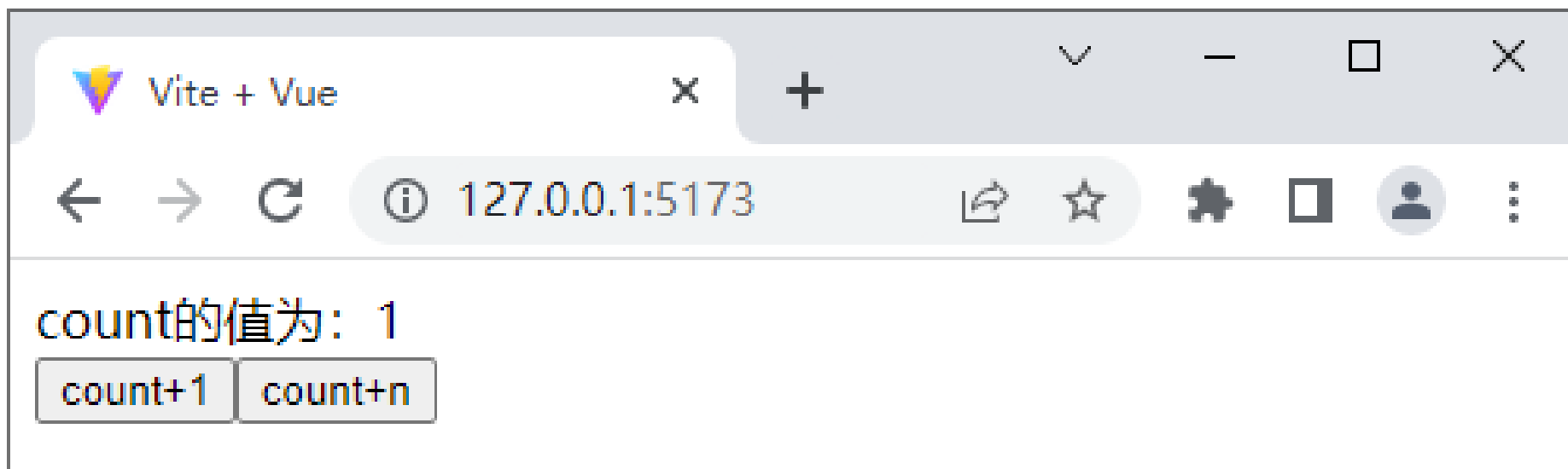
修改页面结构代码，增加一个按钮。

```
<button @click="addCountN(3, $event)">count+n</button>
```

步骤2

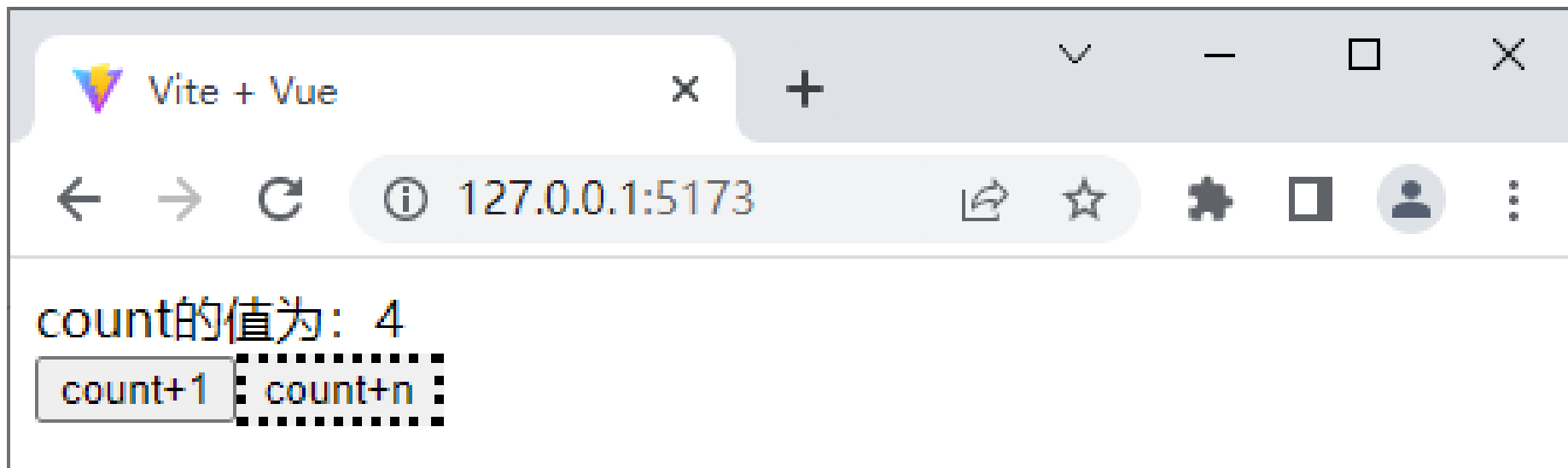
2.4 事件对象

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，通过\$event获取事件对象的页面效果如下图所示。



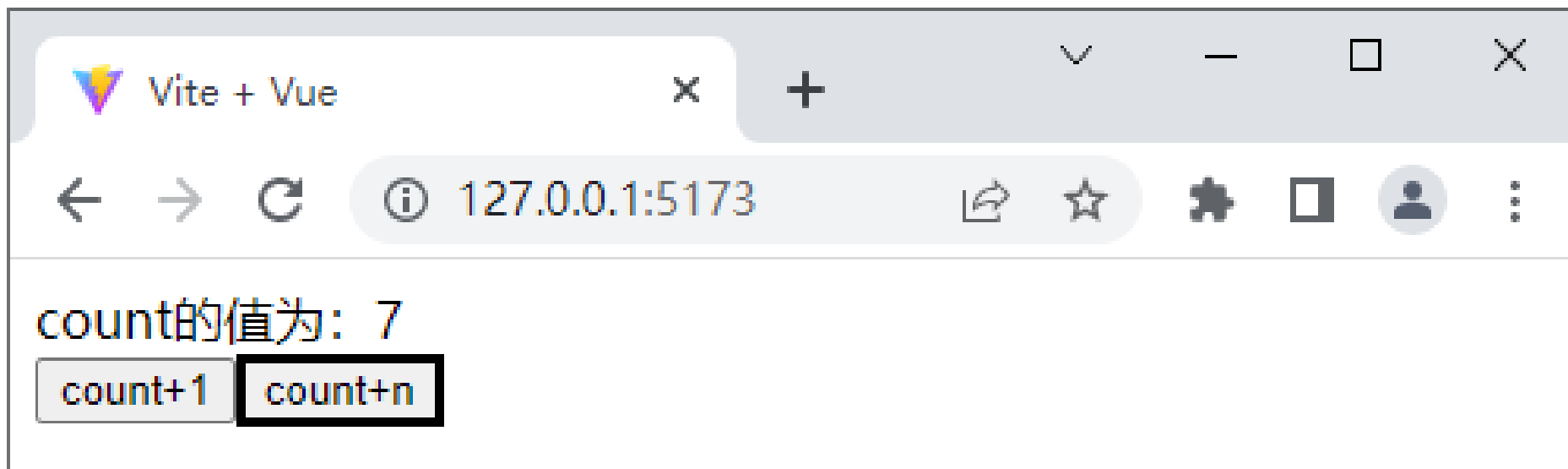
2.4 事件对象

单击 “count+n” 按钮后的页面效果如下图所示。



2.4 事件对象

再次单击 “count+n” 按钮后的页面效果如下图所示。



2.5

事件修饰符

2.5 事件修饰符



熟悉事件修饰符，能够阐述常见的事件修饰符

2.5 事件修饰符

什么是事件修饰符？



2.5 事件修饰符



为了简化开发，Vue为开发者提供了事件修饰符，它可以与v-on配合使用，以便于对事件进行控制，让开发者更专注于逻辑。事件修饰符用于修饰事件的行为，写在事件名称之后，多个事件修饰符可以串联使用。

2.5 事件修饰符

通过事件修饰符可以实现的一些功能。



- 阻止默认事件行为
- 阻止事件冒泡
- 事件捕获
- 使事件只触发一次
- 使DOM元素只有自身触发事件时才执行事件方法
- 监听滚动事件
- 捕获特定按键
- 捕获鼠标按键

2.5 事件修饰符

1. 阻止默认事件行为

通过`.prevent`事件修饰符可以实现阻止默认事件行为的功能。例如，在单击`<a>`标签时页面会自动跳转，这就是`<a>`标签的默认事件行为。

在实际开发中，如果默认事件行为与事件发生冲突，可以使用`.prevent`事件修饰符阻止默认事件行为，示例代码如下。

```
<a href="test.html" v-on:click.prevent>阻止默认行为</a>
```

2.5 事件修饰符

2. 阻止事件冒泡

通过`.stop`事件修饰符可以实现阻止事件冒泡的功能，示例代码如下。

```
<template>
  <div v-on:click="show('我是父元素的事件')">
    <button v-on:click="show('我是子元素的事件')">事件冒泡</button>
    <button v-on:click.stop="show('我是子元素的事件')">阻止事件冒泡
  </button>
</div>
</template>
<script setup>
let show = message => console.log(message)
</script>
```


2.5 事件修饰符

3. 事件捕获

通过`.capture`事件修饰符可以实现事件捕获的功能，示例代码如下。

```
<template>
  <div v-on:click.capture="show('我是父元素的事件')">
    <button v-on:click="show('我是子元素的事件')">事件捕获</button>
  </div>
</template>
<script setup>
let show = message => console.log(message)
</script>
```

2.5 事件修饰符

4. 使事件只触发一次

通过`.once`事件修饰符可以实现使事件只触发一次的功能。`.once`事件修饰符用于阻止事件的多次触发，让事件只触发一次，示例代码如下。

```
<template>
  <button v-on:click.once = "show('我是当前元素的单击事件且只执行一次')">只执行一次</button>
</template>
<script setup>
let show = message => console.log(message)
</script>
```

2.5 事件修饰符

5. 使DOM元素只有自身触发事件时才执行事件方法

通过`.self`事件修饰符可以实现只有DOM元素自身触发事件时才执行事件方法的功能，示例代码如下。

```
<template>
  <div v-on:click="show('我是祖先元素的事件')">祖先元素
    <div v-on:click.self="show('我是父元素的事件')">父元素
      <div v-on:click="show('我是子元素的事件')">子元素</div>
    </div>
  </div>
</template>
<script setup>
let show = message => console.log(message)
</script>
```

2.5 事件修饰符

6. 监听滚动事件

通过`.passive`事件修饰符可以实现监听滚动事件的功能。`.passive`事件修饰符主要用于优化移动端设备的滚屏性能。添加`.passive`事件修饰符后会优先响应滚动事件而不是滚动事件的回调函数，从而可提升性能。

使用`.passive`事件修饰符的示例代码如下。

```
<div v-on:scroll.passive="onScroll"></div>
```

2.5 事件修饰符

7. 捕获特定按键

Vue提供了一些用于修饰键盘事件的修饰符，使用它们可以捕获特定按键，其中常用的修饰符如下。

- `.enter`：捕获Enter键。
- `.esc`：捕获Esc键。
- `.tab`：捕获Tab键。
- `.delete`：捕获Delete和Backspace键。
- `.ctrl`：捕获Ctrl键。
- `.alt`：捕获Alt键。
- `.shift`：捕获Shift键。
- `.meta`：在MacOS系统的键盘上捕获command键；在Windows系统的键盘上捕获Windows徽标键。

2.5 事件修饰符

接下来以捕获Enter键为例，演示.enter事件修饰符的使用，示例代码如下。

```
<template>
  <input type="text" v-on:keyup.enter="submit">
</template>
<script setup>
let submit = () => console.log('捕获到Enter键')
</script>
```

2.5 事件修饰符

如果想实现**只有按下Enter键时才能触发事件**，可以通过**.exact修饰符**来实现，该修饰符允许控制由精确的系统修饰符组合触发的事件。

修改input元素实现**仅有Enter键被按下时才触发**，示例代码如下。

```
<input type="text" v-on:keyup.enter.exact="submit">
```

2.5 事件修饰符

8. 捕获鼠标按键

Vue中提供了一些[用于捕获鼠标按键的事件修饰符](#)，其中常用的修饰符如下。

- [.left](#)：捕获鼠标左键。
- [.middle](#)：捕获鼠标中键。
- [.right](#)：捕获鼠标右键。

2.5 事件修饰符

接下来以捕获鼠标左键为例，演示鼠标按键修饰符的使用，示例代码如下。

```
<template>
  <button v-on:click.left="show('捕获到鼠标左键')">按钮</button>
</template>
<script setup>
let show = message => console.log(message)
</script>
```

2.6

计算属性

2.6 计算属性



掌握**计算属性**，能够灵活运用计算属性
实时监听数据的变化，当所依赖的数据
发生变化时重新计算值

2.6 计算属性

什么是计算属性？



2.6 计算属性



Vue提供了**计算属性**来描述**依赖响应式数据**的复杂逻辑。

计算属性可以**实时监听数据的变化**，返回一个**计算后的新值**，并将**计算结果缓存起来**。

只有计算属性中依赖的**数据源**变化了，**计算属性**才会**自动重新求值**，并重新加入缓存。

2.6 计算属性

在组件中使用计算属性，分为以下2个步骤。



2.6 计算属性

1. 定义计算属性

计算属性通过`computed()`函数定义，该函数的参数为一个回调函数，开发者需要在回调函数中实现计算功能，并在计算完成后返回计算后的数据，语法格式如下。

```
<script setup>
import { computed } from 'vue'
const 计算属性名 = computed(() => {
  return 计算后的数据
})
</script>
```

2.6 计算属性

2. 输出计算属性

将计算属性定义好后，可以使用Mustache语法输出计算属性，语法格式如下。

```
{{ 计算属性名 }}
```


2.6 计算属性

演示计算属性的使用方法

步骤1

创建src\components\Computed.vue文件。

```
<template>  
  <p>初始message: {{ message }}</p>  
  <p>反转之后的message: {{ reversedMessage }}</p>  
  <button @click="updateMessage">更改</button>  
</template>
```

步骤2

2.6 计算属性

演示计算属性的使用方法

步骤1

>> 接上页代码

```
<script setup>
import { ref, computed } from 'vue'
const message = ref('Hello World')
const reversedMessage = computed(() =>
  message.value.split('').reverse().join('')
)
const updateMessage = () => {
  message.value = 'hello'
}
</script>
```

步骤2

2.6 计算属性

演示计算属性的使用方法

步骤1

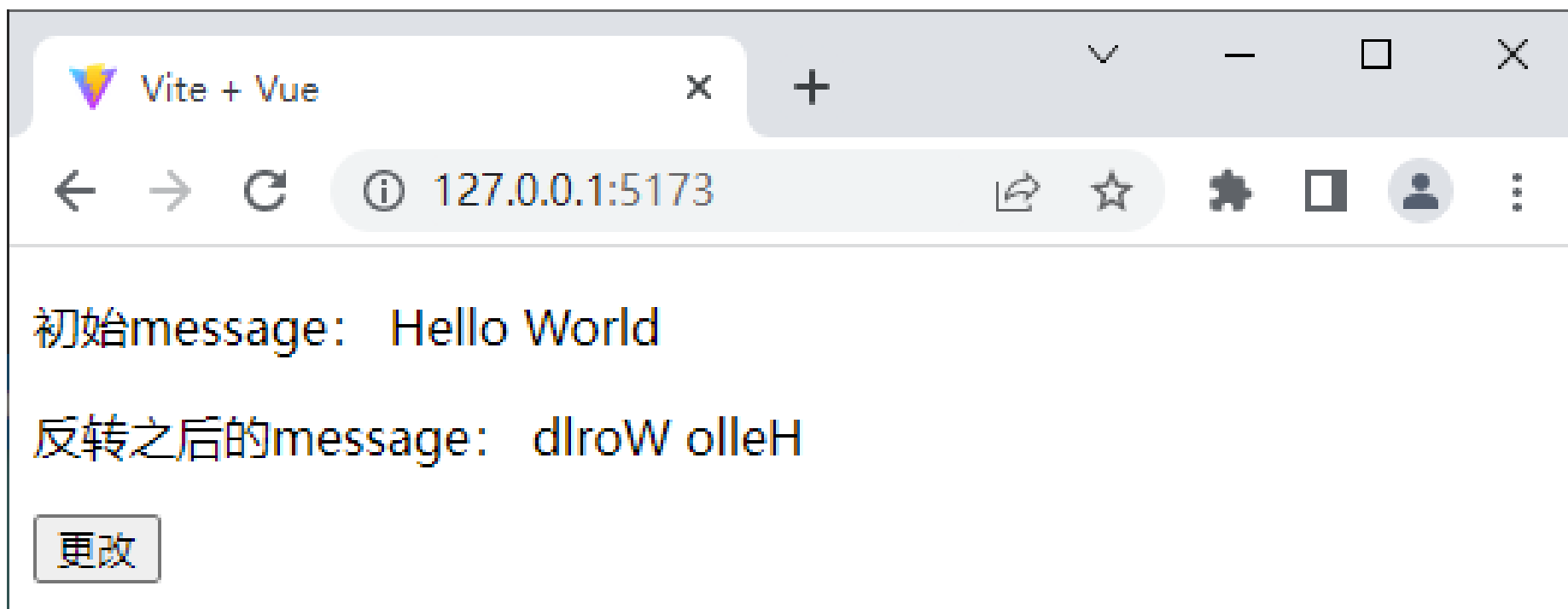
修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

```
import App from './components/Computed.vue'
```

步骤2

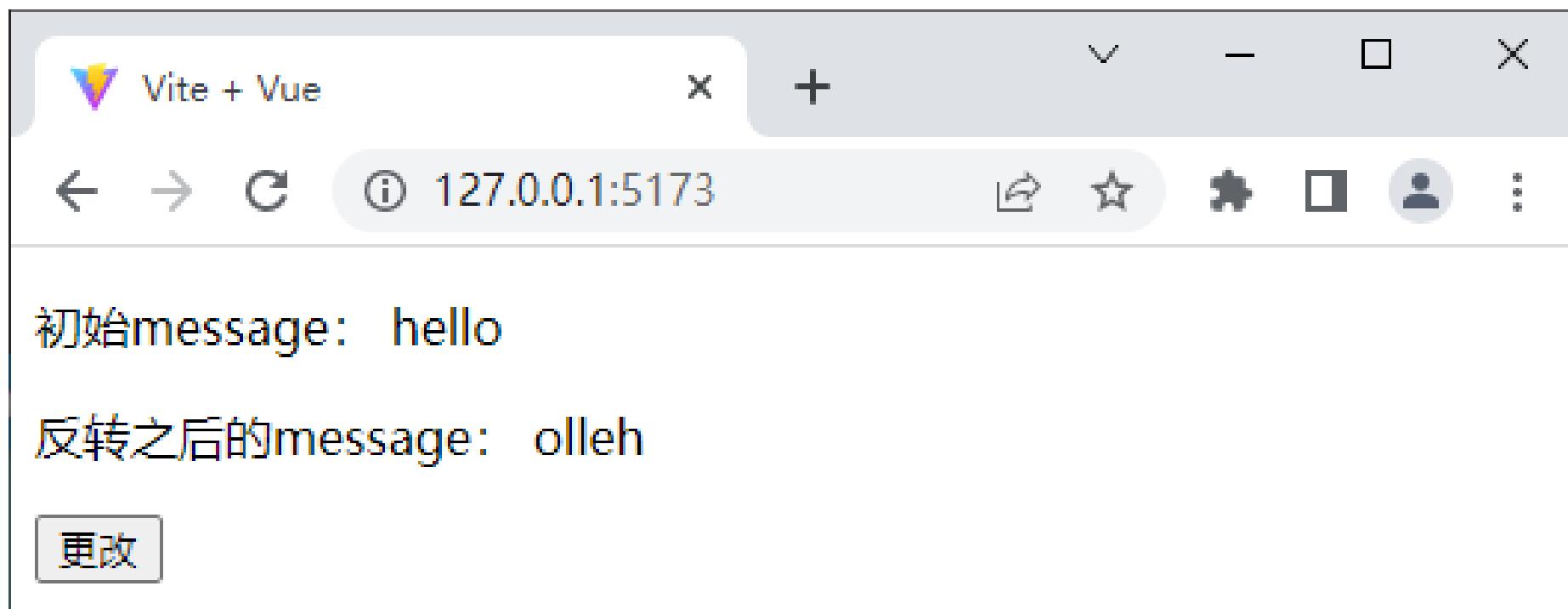
2.6 计算属性

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，通过计算属性实现的页面效果如下图所示。



2.6 计算属性

单击“更改”按钮后的页面效果如下图所示。



2.7

侦听器

2.7 侦听器



掌握**侦听器**，能够灵活运用侦听器监听数据的变化并进行相应的操作

2.7 侦听器

什么是侦听器?



2.7 侦听器



在Vue中，开发者可以[自定义方法](#)来进行数据的更新操作，但是不能自动监听数据的状态。如果想[在数据更新后进行相应的操作](#)，可以通过[侦听器](#)来实现。

2.7 侦听器

侦听器通过watch()函数定义，watch()函数的语法格式如下。

```
watch(侦听器的来源, 回调函数, 可选参数)
```

watch()函数有3个参数，下面对这3个参数分别进行讲解。

第1个参数是侦听器的来源，侦听器的来源可以有以下4种。

- 一个函数，返回一个值。
- 一个响应式数据。
- 一个响应式对象。
- 一个由以上类型的值组成的数组。

2.7 侦听器

第2个参数是数据发生变化时要调用的回调函数，这个回调函数的第1个参数表示新值，即数据发生变化后的值，第2个参数表示旧值，即数据发生变化前的值。

第3个参数是可选参数，它是一个对象，该对象有以下2个常用选项。

- **deep**：默认情况下，当侦听一个对象时，如果对象中的属性值发生了变化，则无法被监听到。如果想监听到，可以将该选项设为true，表示进行深度监听。该选项的默认值为false，表示不使用该选项。
- **immediate**：默认情况下，组件在初次加载完毕后不会调用侦听器的回调函数，如果想让侦听器的回调函数立即被调用，则需要将选项设为true。该选项的默认值为false，表示不使用该选项。

2.7 侦听器

演示侦听器的使用方法

步骤1

创建src\components\Watch.vue文件。

步骤2

```
<template>
  <input type="text" v-model="cityName">
</template>
<script setup>
import { watch, ref } from 'vue'
const cityName = ref('beijing')
watch(cityName, (newVal, oldVal) => {
  console.log(newVal, oldVal)
})
</script>
```

2.7 侦听器

演示侦听器的使用方法

步骤1

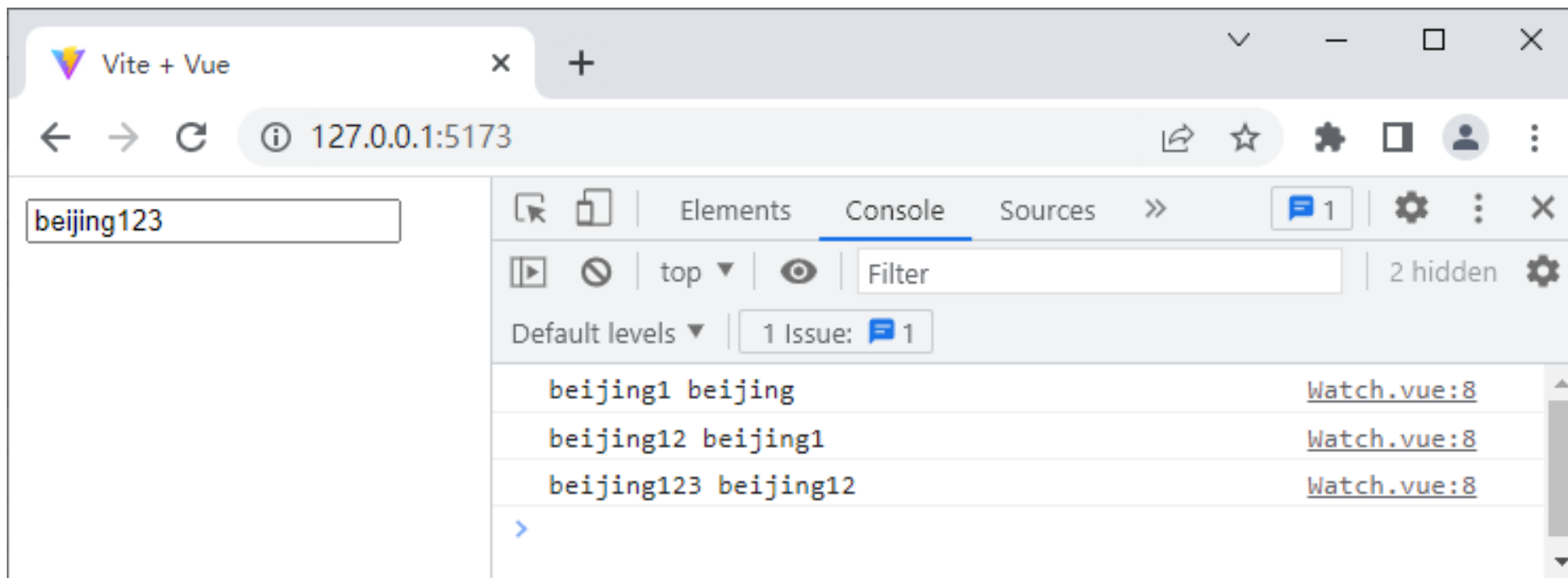
修改src\main.js文件，**切换**页面中**显示**的**组件**。

```
import App from './components/Watch.vue'
```

步骤2

2.7 侦听器

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，在输入框中输入“beijing123”后的页面效果如下图所示。



A blue parallelogram graphic with a dark gray triangle at its top-right corner, containing the white text '2.8'.

2.8

样式绑定

2.8.1 绑定class属性



掌握样式绑定，能够灵活运用v-bind
绑定class属性实现元素样式的设置

2.8.1 绑定class属性



在Vue中，通过更改元素的class属性可以更改元素的样式，而对class属性的操作可以通过v-bind来实现。

使用v-bind绑定class属性时，可以将class属性值绑定为字符串、对象或数组，下面分别进行讲解。

2.8.1 绑定class属性

1. 将class属性值绑定为字符串

在Vue中，可以将class属性值绑定为字符串，示例代码如下。

```
<template>
  <div v-bind:class="className"> </div>
</template>
<script setup>
const className = 'box'
</script>
```

当上述代码运行后，div元素的渲染结果如下。

```
<div class="box"> </div>
```

2.8.1 绑定class属性

演示如何为class属性绑定字符串

步骤1

创建src\components\ClassStr.vue文件。

步骤2

```
<template>
  <div v-bind:class="className">梦想</div>
</template>
<script setup>
const className = 'box'
</script>
<style>
.box {
  border: 1px solid black;
}
</style>
```

2.8.1 绑定class属性

演示如何为class属性绑定字符串

步骤1

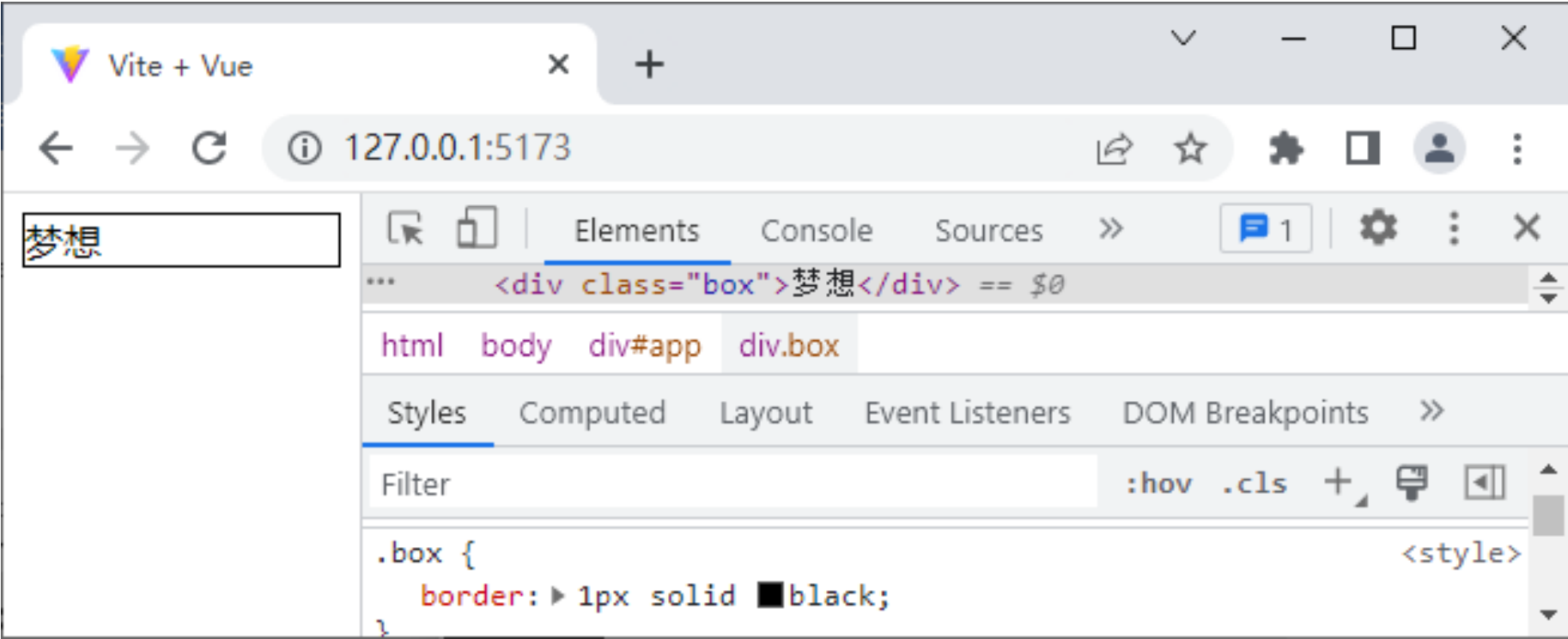
修改src/main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

```
import App from './components/ClassStr.vue'
```

步骤2

2.8.1 绑定class属性

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，为class属性绑定字符串的页面效果如下图所示。



2.8.1 绑定class属性

2. 将class属性值绑定为对象

在Vue中，可以将class属性值绑定为对象，从而动态地改变class属性值。对象中包含的属性名表示类名，属性值为布尔类型，如果属性值为true，表示类名生效，否则类名不生效。将class属性值绑定为对象的示例代码如下。

```
<template>
  <div v-bind:class="{ className: isActive }"> </div>
</template>
<script setup>
import { ref } from 'vue'
const isActive = ref(true)
</script>
```

2.8.1 绑定class属性

运行上述代码后，div元素的渲染结果如下。

```
<div class="className"> </div>
```

使用v-bind绑定的class属性可以与普通的class属性共存，示例代码如下。

```
<div class="className1" v-bind:class="{ className2:  
isActive }"> </div>
```

在上述代码中，当isActive为true时，div元素的渲染结果如下。

```
<div class="className1 className2"> </div>
```

2.8.1 绑定class属性

在使用v-bind绑定class属性时，如果不想将对象类型的class属性值直接写在模板中，可以将属性值定义成一个[响应式对象](#)或一个返回对象的[计算属性](#)，示例代码如下。

```
<template>
  <div v-bind:class="classObject1"> </div>
  <div v-bind:class="classObject2"> </div>
</template>
<script setup>
import { reactive, ref, computed } from 'vue'
const classObject1 = reactive({ className: true })
const isActive = ref(true)
const classObject2 = computed(() => ({
  className: isActive.value
}))
</script>
```


2.8.1 绑定class属性

演示如何为class属性绑定对象

创建src\components\ClassObject.vue文件。

步骤1

```
<template>
  <div class="text" v-bind:class="{ box: isBox, border:
isBorder }">
    <div v-bind:class="{ inner: isInner }">春夏</div>
    <div v-bind:class="classObject">秋冬</div>
    <div v-bind:class="classMeal">三餐四季~</div>
  </div>
</template>
```

步骤2

步骤3

2.8.1 绑定class属性

演示如何为class属性绑定对象

步骤1

>> 接上页代码

步骤2

步骤3

```
<script setup>
import { ref, reactive, computed } from 'vue'
const isBox = ref(true)
const isBorder = ref(true)
const isInner = ref(true)
const isMeal = ref(true)
const classObject = reactive({ inner: true })
const classMeal = computed(() => ({
  meal: isMeal.value
}))
</script>
```

2.8.1 绑定class属性

演示如何为class属性绑定对象

在src\components\ClassObject.vue文件中定义页面所需的样式。

步骤1

步骤2

步骤3

```
<style>
.text {
  text-align: center;
  line-height: 30px;
}
.box {
  width: 100%;
  background: linear-gradient(white, rgb(239, 250, 86));
}
.border { border: 2px dotted black; }
```

2.8.1 绑定class属性

演示如何为class属性绑定对象

步骤1

步骤2

步骤3

>> 接上页代码

```
.inner {  
  margin-top: 2px;  
  width: 100px; height: 30px;  
  border: 2px double black;  
}  
.meal {  
  width: 100px; height: 30px;  
  border: 2px dashed rgb(120, 81, 227);  
}  
</style>
```

2.8.1 绑定class属性

演示如何为class属性绑定对象

步骤1

修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

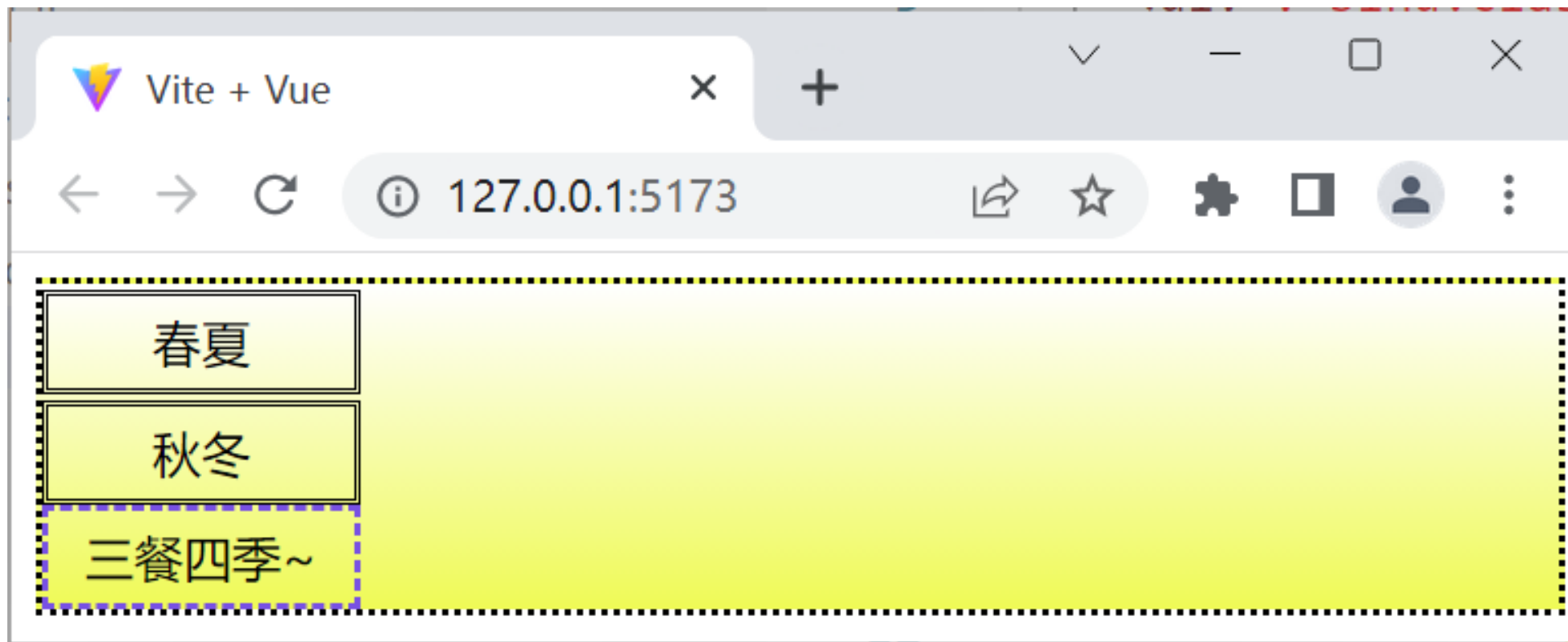
```
import App from './components/ClassObject.vue'
```

步骤2

步骤3

2.8.1 绑定class属性

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，为class属性绑定对象的页面效果如下图所示。



2.8.1 绑定class属性

3. 将class属性值绑定为数组

在Vue中，v-bind指令中的class属性值除了字符串和对象外，还可以是一个数组，用以动态地切换HTML的class属性，数组中的每个元素为要绑定的类名。

将class属性值绑定为数组的示例代码如下。

```
<template>
  <div v-bind:class="[className1, className2]"> </div>
</template>
<script setup>
import { ref } from 'vue'
const className1 = ref('active')
const className2 = ref('error')
</script>
```

2.8.1 绑定class属性

运行上述代码后，div元素的渲染结果如下。

```
<div class="active error"> </div>
```

如果想有条件地切换列表中的class，可以使用三元表达式，示例代码如下。

```
<div v-bind:class="[isActive ? className1 : className2]"> </div>
```

当class有多个条件时，在数组语法中也可以使用对象语法，示例代码如下。

```
<div v-bind:class="{ active: isActive }, className2]"> </div>
```


2.8.1 绑定class属性

演示如何为class属性绑定数组

步骤1

创建src\components\ClassArray.vue文件。

步骤2

步骤3

```
<template>
  <div v-bind:class="[activeClass, borderClass]"> </div>
  <div v-bind:class="[isActive ? activeClass : '',
borderClass]"> </div>
  <div v-bind:class="[ { active: isActive }, borderClass]"> </div>
</template>
<script setup>
import { ref } from 'vue'
const isActive = ref(true)
const activeClass = ref('active')
const borderClass = ref('border')
</script>
```

2.8.1 绑定class属性

演示如何为class属性绑定数组

步骤1

步骤2

步骤3

在src\components\ClassArray.vue文件中定义页面所需的样式。

```
<style>
.active {
  width: 100px;
  height: 10px;
  margin-bottom: 2px;
  background-color: rgb(59, 192, 241);
}
.border {
  border: 2px solid rgb(0, 0, 0);
}
</style>
```

2.8.1 绑定class属性

演示如何为class属性绑定数组

步骤1

修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

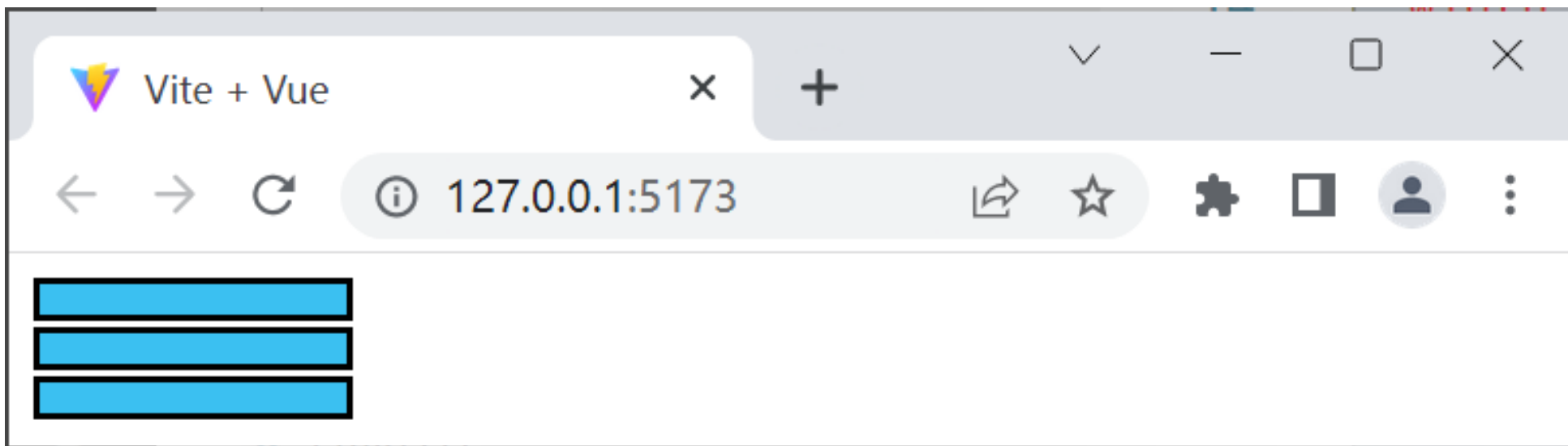
```
import App from './components/ClassArray.vue'
```

步骤2

步骤3

2.8.1 绑定class属性

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，为class属性绑定数组的页面效果如下图所示。



2.8.2 绑定style属性



掌握样式绑定，能够灵活运用v-bind
绑定style属性实现元素样式的设置

2.8.2 绑定style属性



在Vue中，通过更改元素的style属性也可以更改元素的样式，对style属性的操作也可以通过v-bind来实现。使用v-bind绑定style属性时，可以将style属性值绑定为对象或数组，下面分别进行讲解。

2.8.2 绑定style属性

1. 将style属性值绑定为对象

在Vue中，将style属性值绑定为对象时，该对象中的属性名表示CSS属性名，属性值为CSS属性值。以对象语法绑定元素的style属性，示例代码如下。

```
<template>
  <div v-bind:style="{ color: 'red', 'font-size': '30px' }"> </div>
  <div v-bind:style="{ color: activeColor, fontSize: fontSize + 'px' }">
</div>
</template>
<script setup>
import { ref } from 'vue'
const activeColor = ref('red')
const fontSize = ref(30)
</script>
```

2.8.2 绑定style属性

除了将对象直接定义在模板中，还可以将对象定义在<script>标签中，示例代码如下。

```
<template>
  <div v-bind:style="styleObject"> </div>
</template>
<script setup>
import { reactive } from 'vue';
const styleObject = reactive({
  color: 'red',
  fontSize: '12px'
})
</script>
```


2.8.2 绑定style属性

演示如何为style属性绑定对象

步骤1

创建src\components\StyleObject.vue文件。

步骤2

```
<template>
  <!-- 绑定样式属性值 -->
  <div v-bind:style="{ 'background-color': pink, width: width, height:
height + 'px' }">
    <!-- 三元表达式 -->
    <div v-bind:style="{ backgroundColor: isBlue ? blue : 'black',
width: '50px', height: '20px' }"> </div>
    <!-- 绑定样式对象 -->
    <div v-bind:style="myDiv"> </div>
  </div>
</template>
```

2.8.2 绑定style属性

演示如何为style属性绑定对象

步骤1

>> 接上页代码

步骤2

```
<script setup>
import { ref, reactive } from 'vue'
const isBlue = ref(false)
const blue = ref('blue')
const pink = ref('pink')
const width = ref('100%')
const height = ref(40)
const myDiv = reactive({
  backgroundColor: 'red', width: '50px', height: '20px'
})
</script>
```

2.8.2 绑定style属性

演示如何为style属性绑定对象

步骤1

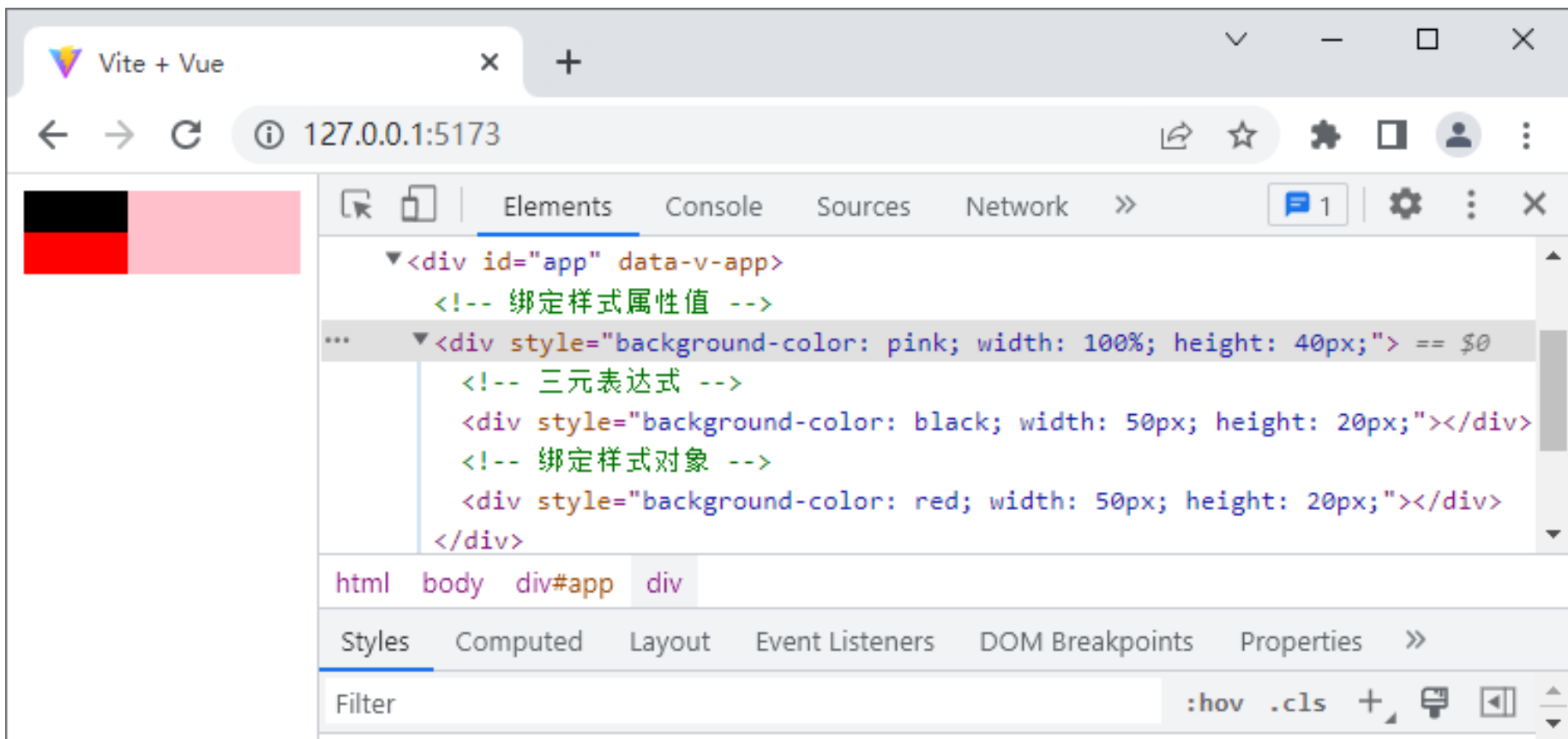
修改src\main.js文件，**切换**页面中**显示**的**组件**。

```
import App from './components/StyleObject.vue'
```

步骤2

2.8.2 绑定style属性

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，为style属性绑定对象的页面效果如下图所示。



2.8.2 绑定style属性

2. 将style属性绑定为数组

将style属性绑定为数组，可以实现将多个样式应用到同一个元素上，示例代码如下。

```
<div v-bind:style="[classObj1, classObj2]"> </div>
```

在上述代码中，数组中的每一个元素都是一个对象，例如classObj1和classObj2，对象中包含CSS属性名和属性值。每个对象需要在<script>标签中定义。

2.8.2 绑定style属性

演示如何为class属性绑定数组

步骤1

创建src\components\StyleArray.vue文件。

步骤2

```
<template>
  <!-- 使用数组 -->
  <div v-bind:style="[activeClass, borderClass]"> </div>
  <!-- 使用三元表达式 -->
  <div v-bind:style="[isActive ? activeClass : '', borderClass]"> </div>
  <!-- 数组语法中使用对象语法 -->
  <div v-bind:style="[{ backgroundColor: 'rgb(59, 192, 241)', height:
'10px' }, borderClass]"> </div>
</template>
```

2.8.2 绑定style属性

演示如何为class属性绑定数组

步骤1

>> 接上页代码

步骤2

```
<script setup>
import { ref, reactive } from 'vue'
const isActive = ref(true)
const activeClass = reactive({
  height: '10px',
  backgroundColor: 'rgb(59, 192, 241)'
})
const borderClass = reactive({
  border: '2px solid rgb(0, 0, 0)'
})
</script>
```

2.8.2 绑定style属性

演示如何为class属性绑定数组

步骤1

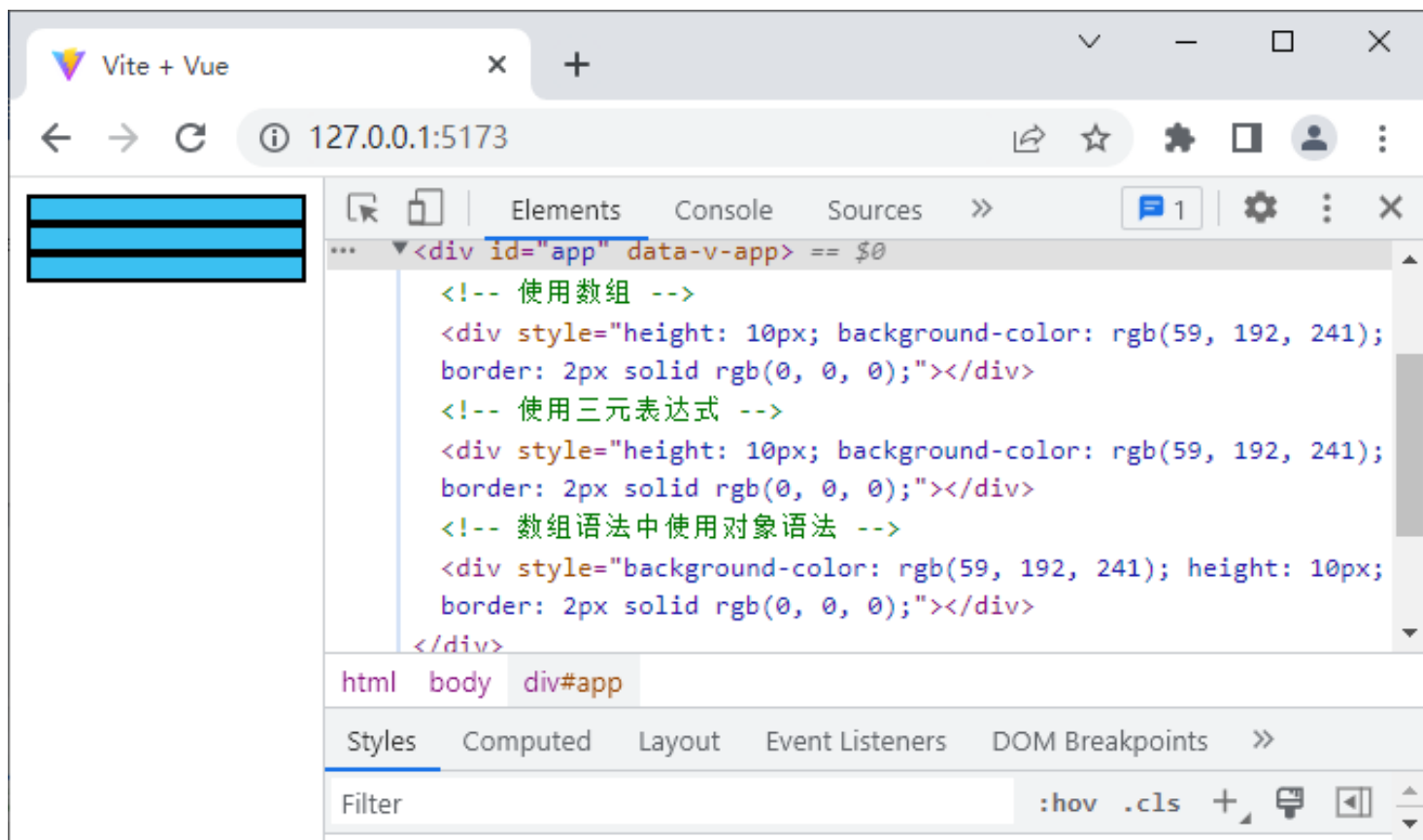
修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

```
import App from './components/StyleArray.vue'
```

步骤2

2.8.2 绑定style属性

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，为style属性绑定数组的页面效果如下图所示。



2.9

阶段案例——学习计划表

2.9 阶段案例——学习计划表



掌握学习计划表，能够实现学习计划表的开发

2.9 阶段案例——学习计划表



“学习计划表”用于对学习计划进行管理，包括对学习计划进行添加、删除、修改等操作。

2.9 阶段案例——学习计划表

1. 初始页面效果展示

当“学习计划表”案例打开后，页面中会展示学生的学习计划，包括学习科目、学习内容、学习地点、完成状态等。



2.9 阶段案例——学习计划表

1. 初始页面效果展示

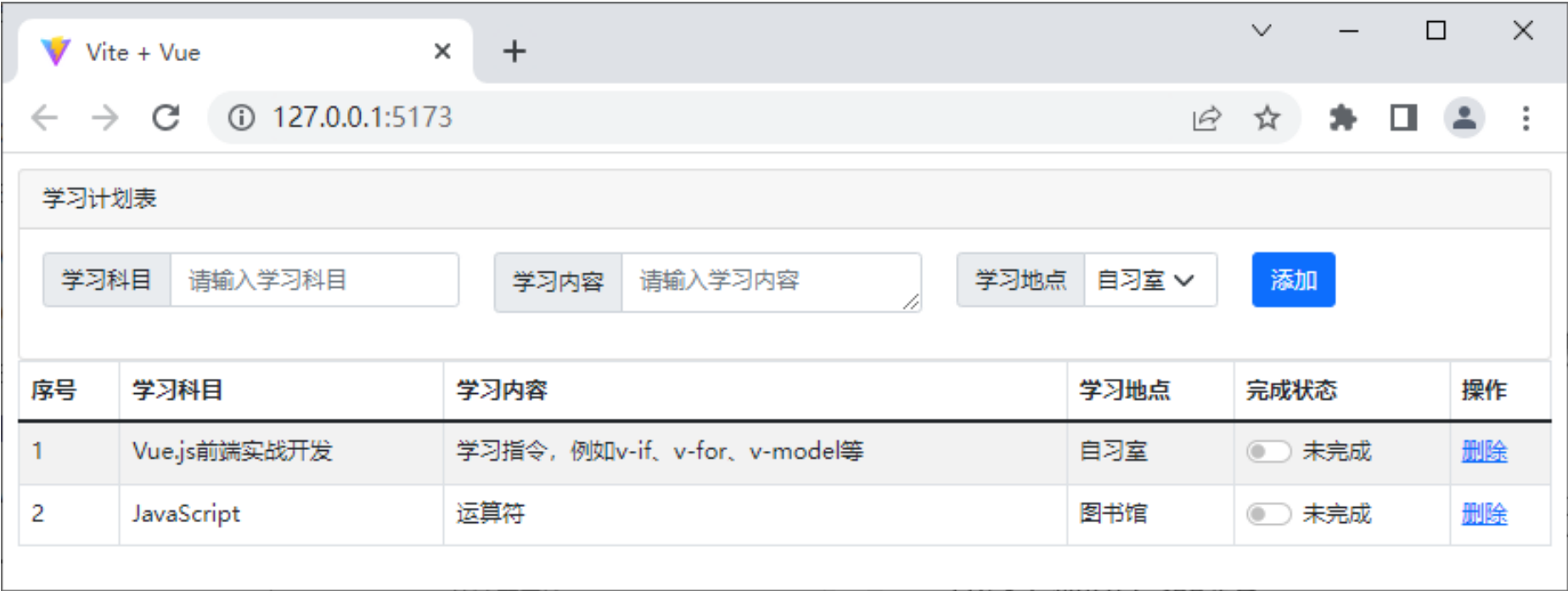
页面上半部分为卡片区域，包含标题区域和表单区域。其中，标题区域中展示标题为“学习计划表”，表单区域中可以根据实际情况输入学习科目、学习内容、学习地点等内容，学习地点有3个选项，包括自习室、图书馆、宿舍，单击“添加”按钮即可添加学习计划，默认添加完成状态为“未完成”。

页面下半部分为表格区域，用于展示学习计划列表。在表单区域中添加并提交的信息会在表格区域中展示。学生可以在表格区域更改学习计划的完成状态为“已完成”或者“未完成”。

2.9 阶段案例——学习计划表

2. 添加学习计划页面效果展示

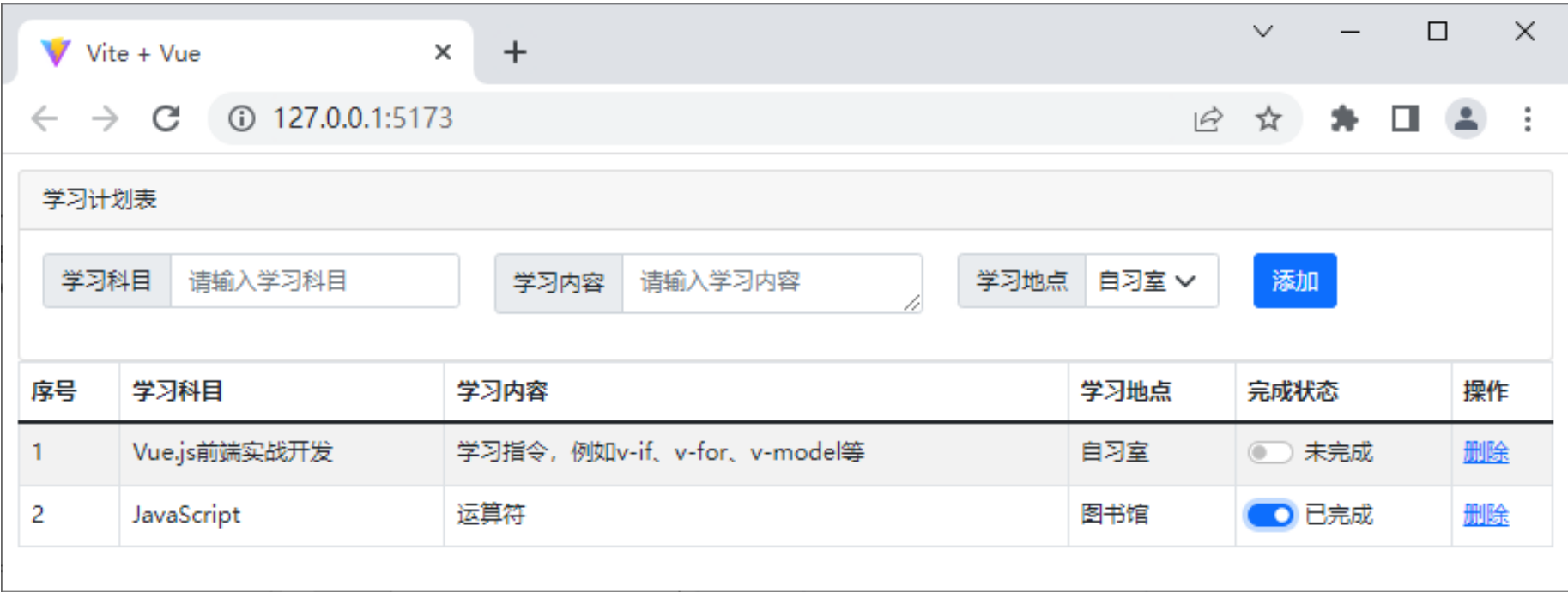
在表单区域中输入学习计划，学习科目为“JavaScript”，学习内容为“运算符”，学习地点为“图书馆”，单击“添加”按钮之后的页面效果如下图所示。



2.9 阶段案例——学习计划表

3. 修改学习计划完成状态的页面效果展示

将学习计划2的完成状态更改为“**已完成**”的页面效果如下图所示。



2.9 阶段案例——学习计划表

4. 删除学习计划

当学习计划处于“已完成”状态时，学生可以对学习计划进行删除操作，否则不能进行删除操作。

本章小结

本章主要讲解了Vue的基础知识，内容主要包括单文件组件、数据绑定、指令、事件对象、事件修饰符、计算属性、侦听器和样式绑定，最后运用本章所学知识完成“学习计划表”案例的开发。通过本章的学习，读者应掌握Vue的基本语法，能够使用Vue完成一些简单程序的编写。